

ĐẠI HỌC BÁCH KHOA HÀ NỘI

ĐỒ ÁN TỐT NGHIỆP

**Ứng dụng di động chấm thi trắc nghiệm di động tích
hợp OCR và Generative AI**

Đinh Mạnh Dũng

mahndugn@gmail.com

Chương trình đào tạo: Khoa học máy tính

SĐT: 0776228213

Lớp: IT4995 - Mã lớp: 762000

MSSV: 20224959

Giảng viên hướng dẫn: TS. Trần Vĩnh Đức

Khoa: Kỹ thuật máy tính

Trường: Công nghệ thông tin và Truyền thông

HÀ NỘI, 06/2026

ĐẠI HỌC BÁCH KHOA HÀ NỘI

ĐỒ ÁN TỐT NGHIỆP

**Ứng dụng di động chấm thi trắc nghiệm di động tích
hợp OCR và Generative AI**

Đinh Mạnh Dũng

mahndugn@gmail.com

Chương trình đào tạo: Khoa học máy tính

SĐT: 0776228213

Lớp: IT4995 - Mã lớp: 762000

MSSV: 20224959

Giảng viên hướng dẫn: TS. Trần Vĩnh Đức _____

Chữ kí GVHD

Khoa: Kỹ thuật máy tính

Trường: Công nghệ Thông tin và Truyền thông

HÀ NỘI, 06/2026

Trước hết, em xin bày tỏ lòng biết ơn sâu sắc nhất tới thầy Trần Vĩnh Đức, người đã tận tình hướng dẫn, định hướng và truyền đạt những kiến thức quý báu cho em trong suốt quá trình thực hiện đồ án tốt nghiệp. Thầy không chỉ giúp em về mặt chuyên môn mà còn là người thầy đáng kính trong suốt hành trình học tập tại Đại học Bách khoa Hà Nội.

Em xin gửi lời cảm ơn chân thành tới các thầy cô trong Khoa Kỹ thuật máy tính, Trường Công nghệ thông tin và Truyền thông, Đại học Bách khoa Hà Nội, những người đã trang bị cho em nền tảng kiến thức vững chắc trong suốt bốn năm học tại ngôi trường này.

Em cũng xin cảm ơn gia đình và bạn bè, những người đã luôn đồng hành, động viên và tạo mọi điều kiện tốt nhất để em có thể tập trung hoàn thành đồ án.

Cuối cùng, em xin ghi nhận sự nỗ lực và cố gắng của chính bản thân mình trong suốt quá trình nghiên cứu và phát triển ứng dụng, dù còn nhiều hạn chế nhưng đây là thành quả em đã dành trọn tâm huyết để hoàn thành.

TÓM TẮT NỘI DUNG ĐỒ ÁN

Trong bối cảnh giáo dục hiện đại, việc số hóa và tự động hóa quy trình kiểm tra đánh giá đang trở thành xu hướng tất yếu, đặc biệt khi hình thức thi trắc nghiệm được áp dụng rộng rãi ở các bậc học. Tuy nhiên, phương thức chấm thi trắc nghiệm truyền thống bằng tay vẫn tồn tại nhiều hạn chế nghiêm trọng: tốn kém thời gian, dễ phát sinh sai sót do yếu tố con người, và yêu cầu chi phí đầu tư cho thiết bị chuyên dụng. Trên thị trường hiện nay đã có một số ứng dụng hỗ trợ chấm thi trắc nghiệm, song phần lớn mới chỉ dừng lại ở việc đọc điểm số mà chưa cung cấp các phân tích chuyên sâu có giá trị cho giáo viên.

Đồ án này tập trung nghiên cứu và xây dựng ứng dụng di động SMART GRADING, cho phép giáo viên sử dụng camera trên thiết bị di động thông minh để quét và chấm điểm phiếu trả lời trắc nghiệm một cách tự động. Hệ thống tích hợp kỹ thuật Nhận dạng ký tự quang học (OCR) để trích xuất đáp án từ ảnh chụp phiếu thi, kết hợp với các giải thuật xử lý ảnh nâng cao như phát hiện góc tài liệu, căn chỉnh phối cảnh và tách vùng bọt đáp án. Bên cạnh đó, hệ thống còn tích hợp Generative AI để phân tích kết quả bài thi, tự động tổng hợp phổ điểm, nhận diện các câu hỏi học sinh thường làm sai và đưa ra nhận xét chi tiết, giúp giáo viên có cái nhìn toàn diện và điều chỉnh phương pháp giảng dạy kịp thời.

Kết quả thực nghiệm cho thấy ứng dụng hoạt động ổn định trên thiết bị thực tế, đạt độ chính xác cao trong nhận dạng đáp án và tích hợp AI mang lại giá trị phân tích vượt trội so với các giải pháp chấm thi thông thường trên thị trường.

Đinh Mạnh Dũng
(Ký và ghi rõ họ tên)

ABSTRACT

In the context of modern education, digitizing and automating assessment processes has become an inevitable trend, especially as multiple-choice testing is widely adopted across all educational levels. However, traditional manual multiple-choice grading still presents significant drawbacks: time-consuming procedures, error-prone due to human factors, and requiring costly specialized equipment. Although several multiple-choice grading applications exist in the market, most only provide basic score reporting without offering in-depth analytical insights for teachers.

This thesis focuses on researching and building the SMART GRADING mobile application, which enables teachers to use smartphone cameras to automatically scan and grade multiple-choice answer sheets. The system integrates Optical Character Recognition (OCR) techniques to extract answers from photographed answer sheets, combined with advanced image processing algorithms such as document corner detection, perspective transformation, and answer bubble segmentation. Additionally, the system incorporates Generative AI to analyze exam results, automatically generate score distribution summaries, identify frequently missed questions, and provide detailed feedback, offering teachers comprehensive insights to adjust their teaching methods promptly.

Experimental results demonstrate that the application operates stably on real devices, achieves high accuracy in answer recognition, and the AI integration delivers superior analytical value compared to conventional multiple-choice grading solutions available on the market.

Dinh Mạnh Dũng
(Signature and Full Name)

MỤC LỤC

CHƯƠNG 1. GIỚI THIỆU ĐỀ TÀI	1
1.1 Tổng quan.....	1
Tổng quan	1
1.2 Đặt vấn đề.....	1
1.3 Mục tiêu và phạm vi đề tài	1
1.4 Định hướng giải pháp.....	2
1.5 Bố cục đồ án	2
CHƯƠNG 2. KHẢO SÁT VÀ PHÂN TÍCH YÊU CẦU	4
2.1 Tổng quan.....	4
Tổng quan	4
2.2 Khảo sát hiện trạng	4
2.3 Tổng quan chức năng.....	5
2.3.1 Biểu đồ use case tổng quát	5
2.3.2 Biểu đồ use case phân rã Quét phiếu thi	6
2.3.3 Quy trình nghiệp vụ	7
2.4 Đặc tả chức năng	7
2.4.1 Đặc tả use case Quản lý đề thi	8
2.4.2 Đặc tả use case Quét phiếu thi	9
2.4.3 Đặc tả use case Phân tích kết quả bằng AI.....	10
2.4.4 Đặc tả use case Xem kết quả thi	10
2.5 Yêu cầu phi chức năng.....	11
CHƯƠNG 3. NỀN TẢNG LÝ THUYẾT VÀ CÔNG NGHỆ SỬ DỤNG..	13
3.1 Tổng quan.....	13
Tổng quan	13

3.2 Nền tảng phát triển ứng dụng di động	13
3.2.1 Lựa chọn Flutter	13
3.2.2 Các công cụ và thư viện hỗ trợ trên Flutter	14
3.3 Xử lý ảnh và Thị giác máy tính	14
3.3.1 Khái quát về xử lý ảnh trên di động	14
3.3.2 Các kỹ thuật xử lý ảnh cốt lõi cho bài toán OMR	15
3.4 Kỹ thuật Nhận dạng ký tự quang học (OCR)	16
3.5 Cơ sở dữ liệu và Backend	16
3.6 Generative AI và tích hợp API	17
3.7 Kết chương	18
Kết chương	18
CHƯƠNG 4. PHÂN TÍCH THIẾT KẾ, TRIỂN KHAI VÀ ĐÁNH GIÁ	
HỆ THỐNG	19
4.1 Tổng quan	19
Tổng quan	19
4.2 Thiết kế kiến trúc	19
4.2.1 Lựa chọn kiến trúc phần mềm	19
4.2.2 Thiết kế tổng quan	20
4.2.3 Thiết kế chi tiết gói	21
4.3 Thiết kế chi tiết	23
4.3.1 Thiết kế giao diện	23
4.3.2 Thiết kế cơ sở dữ liệu	24
4.4 Xây dựng ứng dụng	25
4.4.1 Thư viện và công cụ sử dụng	25
4.4.2 Kết quả đạt được	27
4.4.3 Minh họa các chức năng chính	27

4.5 Kiểm thử	28
4.6 Triển khai	29
4.7 Kết chương	30
Kết chương.....	30
CHƯƠNG 5. CÁC GIẢI PHÁP VÀ ĐÓNG GÓP NỔI BẬT	31
5.1 Tổng quan.....	31
Tổng quan	31
5.2 Giải pháp nhận dạng và trích xuất bọt đáp án trên thiết bị di động.....	31
5.2.1 Bài toán đặt ra	31
5.2.2 Giải pháp đề xuất.....	31
5.3 Giải pháp tích hợp Generative AI vào phân tích kết quả thi	32
5.3.1 Bài toán đặt ra	32
5.3.2 Giải pháp đề xuất.....	33
5.4 Giải pháp tối ưu hóa hiệu năng xử lý ảnh trên di động.....	34
5.4.1 Bài toán đặt ra	34
5.4.2 Giải pháp đề xuất.....	34
5.5 Kết chương	34
Kết chương.....	34
CHƯƠNG 6. KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN.....	36
6.1 Kết luận.....	36
6.2 Hướng phát triển	37
TÀI LIỆU THAM KHẢO	39
PHỤ LỤC	40
A. HƯỚNG DẪN VIẾT ĐỒ ÁN TỐT NGHIỆP.....	40
A.1 Ngành học.....	41
A.2 Đánh dấu (bullet) và đánh số (numering)	41

A.3 Cách thêm bảng	42
A.4 Chèn hình ảnh	42
A.5 Tài liệu tham khảo	43
A.6 Cách viết phương trình và công thức toán học	43
A.7 Qui cách đóng quyển.....	43
B. ĐẶC TẢ USE CASE	45
B.1 Đặc tả use case “Thống kê tình hình mượn sách”	45
B.2 Đặc tả use case “Đăng ký làm thẻ mượn”	45

DANH MỤC HÌNH VẼ

Hình 2.1	Biểu đồ use case tổng quát của hệ thống SMART GRADING	6
Hình 2.2	Biểu đồ use case phân rã chức năng Quét phiếu thi	7
Hình 4.1	Sơ đồ kiến trúc tổng thể hệ thống SMART GRADING	20
Hình 4.2	Biểu đồ gói (Package Diagram) của ứng dụng di động	21
Hình 4.3	Biểu đồ trình tự chức năng Quét phiếu thi	22
Hình 4.4	Minh họa thiết kế giao diện các màn hình chính	24
Hình 4.5	Biểu đồ thực thể liên kết của hệ thống SMART GRADING .	24
Hình 4.6	Minh họa chức năng quét phiếu thi và kết quả chấm điểm . .	28
Hình 4.7	Minh họa chức năng phân tích kết quả bằng Generative AI .	28
Hình A.1	Internet vạn vật	42
Hình A.2	Quy cách đóng quyển đồ án	44
Hình A.3	Quy cách đóng quyển đồ án	44

DANH MỤC BẢNG BIỂU

Bảng 2.1	So sánh các sản phẩm chấm thi trắc nghiệm trên thị trường .	5
Bảng 4.1	Danh sách thư viện và công cụ sử dụng	26
Bảng 4.2	Thống kê kết quả xây dựng hệ thống SMART GRADING .	27
Bảng 4.3	Kết quả kiểm thử độ chính xác module OMR	29
Bảng A.1	Table to test captions and labels.	42

DANH MỤC THUẬT NGỮ VÀ TỪ VIẾT TẮT

Thuật ngữ	Ý nghĩa
AI	Artificial Intelligence — Trí tuệ nhân tạo
API	Application Programming Interface — Giao diện lập trình ứng dụng
Backend	Phần phía server của ứng dụng, xử lý logic nghiệp vụ và lưu trữ dữ liệu
BLoC	Business Logic Component — Mô hình kiến trúc quản lý trạng thái ứng dụng
CNN	Convolutional Neural Network — Mạng nơ-ron tích chập
CRUD	Create, Read, Update, Delete — Tạo, Đọc, Sửa, Xóa
Flutter	Framework phát triển ứng dụng di động đa nền tảng của Google
Frontend	Phần giao diện người dùng của ứng dụng
Gemini	Google Gemini — Mô hình AI đa phương thức của Google
GenAI	Generative Artificial Intelligence — Trí tuệ nhân tạo tạo sinh
HTML	HyperText Markup Language — Ngôn ngữ đánh dấu siêu văn bản
iOS	iPhone Operating System — Hệ điều hành di động của Apple
JSON	JavaScript Object Notation — Định dạng đối tượng JavaScript
OCR	Optical Character Recognition — Nhận dạng ký tự quang học
OMR	Optical Mark Recognition — Nhận dạng dấu quang học
REST API	Representational State Transfer API — Giao diện lập trình theo phong cách REST

Thuật ngữ	Ý nghĩa
-----------	---------

CHƯƠNG 1. GIỚI THIỆU ĐỀ TÀI

1.1 Tổng quan

1.2 Đặt vấn đề

Trong bối cảnh giáo dục hiện đại, đặc biệt với sự phổ biến của hình thức thi trắc nghiệm ở các bậc học từ phổ thông đến đại học, việc kiểm tra đánh giá chất lượng học tập của học sinh, sinh viên diễn ra với tần suất cao và quy mô lớn. Hình thức thi trắc nghiệm mang lại nhiều ưu điểm như tính khách quan, đánh giá được số lượng lớn thí sinh trong thời gian ngắn, và dễ dàng chuẩn hóa tiêu chí đánh giá. Tuy nhiên, quy trình chấm thi trắc nghiệm truyền thống vẫn đang đối mặt với những thách thức đáng kể.

Phương pháp chấm thi bằng tay đòi hỏi giáo viên phải đọc và đối chiếu từng phiếu trả lời với đáp án, thao tác lặp đi lặp lại nhiều lần và dễ dẫn đến sai sót do mệt mỏi, nhầm lẫn đáp án hoặc lỗi đánh dấu điểm. Với một lớp có 40 đến 50 học sinh, mỗi bài thi gồm 40 đến 50 câu hỏi, khối lượng công việc chấm thi của giáo viên trở nên rất lớn, tốn nhiều thời gian mà năng suất lại thấp. Ngoài ra, các thiết bị quét chuyên dụng (scanner) phục vụ cho việc chấm thi trắc nghiệm thường có chi phí đầu tư cao, không phải cơ sở giáo dục nào cũng có điều kiện trang bị.

Một số ứng dụng chấm thi trắc nghiệm đã xuất hiện trên thị trường, tuy nhiên phần lớn mới chỉ dừng lại ở mức độ đọc và tính điểm số, chưa cung cấp các phân tích chuyên sâu có giá trị thực tiễn cho giáo viên như nhận diện các dạng câu hỏi học sinh hay sai, đánh giá mức độ khó của đề thi, hay tổng hợp xu hướng điểm số theo thời gian. Đặc biệt, việc tích hợp các công nghệ mới như Generative AI vào quy trình phân tích kết quả thi vẫn còn rất hạn chế.

Từ thực trạng trên, bài toán đặt ra là cần xây dựng một giải pháp vừa giải quyết được bài toán chấm thi tự động, vừa cung cấp các phân tích chuyên sâu dựa trên trí tuệ nhân tạo, giúp giáo viên tiết kiệm thời gian, nâng cao độ chính xác và có được cái nhìn toàn diện về chất lượng học tập của học sinh, sinh viên.

1.3 Mục tiêu và phạm vi đề tài

Trên cơ sở phân tích thực trạng và xác định các hạn chế của phương pháp chấm thi trắc nghiệm hiện nay, đề tài hướng tới việc xây dựng một ứng dụng di động cho phép giáo viên sử dụng camera trên thiết bị thông minh để quét và chấm điểm phiếu trả lời trắc nghiệm một cách tự động, đồng thời tích hợp Generative AI để phân tích kết quả thi một cách thông minh.

Cụ thể, đề tài hướng tới các mục tiêu sau: nghiên cứu và áp dụng kỹ thuật Nhận

dạng ký tự quang học (OCR) và xử lý ảnh để nhận diện và trích xuất đáp án từ ảnh chụp phiếu thi trắc nghiệm; xây dựng module xử lý ảnh trên thiết bị di động với khả năng phát hiện góc tài liệu, căn chỉnh phối cảnh và tách vùng bọt đáp án với độ chính xác cao; phát triển ứng dụng di động đa nền tảng hoàn chỉnh với đầy đủ các chức năng quản lý lớp học, quản lý đề thi, quản lý kết quả thi; tích hợp Generative AI để tự động phân tích kết quả bài thi, tổng hợp phổ điểm, nhận diện các câu hỏi học sinh hay làm sai và đưa ra nhận xét chi tiết.

Phạm vi nghiên cứu và phát triển tập trung vào hệ thống thi trắc nghiệm với định dạng phiếu trả lời chuẩn hóa, hỗ trợ các loại câu hỏi trắc nghiệm bốn hoặc năm phương án lựa chọn. Ứng dụng được phát triển trên nền tảng di động Android và iOS thông qua framework Flutter, sử dụng các thư viện xử lý ảnh như OpenCV và Google ML Kit. Phần phân tích AI được thực hiện thông qua việc tích hợp API Generative AI.

1.4 Định hướng giải pháp

Để giải quyết bài toán đặt ra, đề tài lựa chọn hướng tiếp cận kết hợp giữa thị giác máy tính (Computer Vision) và trí tuệ nhân tạo tạo sinh (Generative AI). Cụ thể, đề tài sử dụng kỹ thuật Nhận dạng ký tự quang học (OCR) kết hợp với các giải thuật xử lý ảnh nâng cao của thị giác máy tính để nhận diện vùng chứa bọt đáp án trên phiếu thi, phân tích mức độ tô đậm của từng bọt để xác định đáp án được chọn, từ đó đối chiếu với đáp án mẫu để tính điểm tự động.

Về nền tảng phát triển, đề tài sử dụng Flutter làm framework phát triển ứng dụng di động đa nền tảng, cho phép xây dựng một codebase duy nhất chạy trên cả Android và iOS, đảm bảo hiệu quả phát triển và tính đồng nhất trải nghiệm người dùng. Flutter được lựa chọn vì khả năng biên dịch sang mã máy gốc (native code), hiệu năng cao đặc biệt phù hợp với các tác vụ xử lý ảnh thời gian thực, hệ sinh thái thư viện phong phú và cộng đồng phát triển lớn mạnh.

Kết quả chính của đề tài là một ứng dụng di động hoàn chỉnh SMART GRADING, đóng vai trò như một công cụ hỗ trợ giáo viên toàn diện trong quy trình kiểm tra đánh giá: từ khâu tạo và quản lý đề thi, quản lý lớp học, quét và chấm thi tự động, cho tới phân tích kết quả chuyên sâu bằng trí tuệ nhân tạo.

1.5 Bố cục đồ án

Phần còn lại của báo cáo đồ án tốt nghiệp này được tổ chức như sau.

Chương 2 trình bày kết quả khảo sát hiện trạng quy trình chấm thi trắc nghiệm hiện nay, phân tích các sản phẩm tương tự trên thị trường, tổng quan các chức năng của hệ thống SMART GRADING, biểu đồ use case, quy trình nghiệp vụ, đặc tả chi

tiết các use case quan trọng và các yêu cầu phi chức năng của hệ thống.

Trong Chương 3, em giới thiệu về các nền tảng lý thuyết và công nghệ được sử dụng trong đề tài bao gồm Flutter, các kỹ thuật xử lý ảnh và thị giác máy tính (OpenCV), kỹ thuật Nhận dạng ký tự quang học (OCR), cùng với Generative AI và cách tích hợp API. Mỗi công nghệ được phân tích theo hướng đánh giá các lựa chọn thay thế và lý do lựa chọn.

Chương 4 trình bày chi tiết về phân tích và thiết kế hệ thống theo kiến trúc đa tầng (MVC/BLoC), thiết kế tổng quan kiến trúc phần mềm, thiết kế giao diện người dùng, thiết kế cơ sở dữ liệu, các kết quả xây dựng và triển khai ứng dụng cùng với các hình ảnh minh họa, kết quả kiểm thử và triển khai thử nghiệm.

Chương 5 trình bày các giải pháp và đóng góp nổi bật của đề tài, bao gồm chi tiết về thuật toán nhận dạng và trích xuất bọt đáp án, giải pháp tích hợp Generative AI vào phân tích kết quả thi, và các cải tiến nổi bật trong quy trình xử lý ảnh trên thiết bị di động.

Chương 6 tổng kết lại toàn bộ nội dung đã thực hiện, so sánh kết quả đạt được với mục tiêu đề ra, đánh giá các ưu điểm và hạn chế của hệ thống, đồng thời đề xuất các hướng phát triển tiếp theo để hoàn thiện và mở rộng ứng dụng trong tương lai.

CHƯƠNG 2. KHẢO SÁT VÀ PHÂN TÍCH YÊU CẦU

2.1 Tổng quan

Chương 1 đã trình bày bối cảnh thực tiễn và sự cần thiết của việc xây dựng ứng dụng chấm thi trắc nghiệm di động tích hợp OCR và Generative AI. Chương 2 này sẽ khảo sát chi tiết hiện trạng quy trình chấm thi trắc nghiệm hiện nay, phân tích và so sánh các sản phẩm tương tự trên thị trường, từ đó xác định các yêu cầu chức năng và phi chức năng cho hệ thống SMART GRADING. Nội dung chương bao gồm khảo sát hiện trạng, tổng quan chức năng với biểu đồ use case và quy trình nghiệp vụ, đặc tả chi tiết các use case quan trọng, và cuối cùng là các yêu cầu phi chức năng.

2.2 Khảo sát hiện trạng

Hiện tại, quy trình chấm thi trắc nghiệm tại các trường học ở Việt Nam được thực hiện chủ yếu qua ba phương thức. Phương thức phổ biến nhất là chấm bằng tay, trong đó giáo viên sử dụng đáp án để đối chiếu từng phiếu trả lời của học sinh, đánh dấu kết quả và tính điểm. Phương thức thứ hai là sử dụng máy quét OMR chuyên dụng, các thiết bị này có khả năng đọc các vòng tròn đã tô trên phiếu trắc nghiệm và xuất kết quả ra máy tính, tuy nhiên chi phí đầu tư cho một máy quét chất lượng dao động từ vài triệu đến hàng chục triệu đồng. Phương thức thứ ba là sử dụng phần mềm trên máy tính kết hợp với máy scan, yêu cầu cơ sở vật chất phòng máy và thiết bị scan.

Quy trình chấm thi bằng tay tồn tại nhiều hạn chế nghiêm trọng. Thứ nhất, vấn đề về thời gian và công sức: với một lớp 45 học sinh, mỗi bài thi 50 câu, giáo viên cần thực hiện khoảng 2.250 phép so sánh đáp án, công việc lặp đi lặp lại và nhàm chán. Thứ hai, sai sót do yếu tố con người: mệt mỏi dẫn đến nhầm lẫn dòng, sót câu hoặc đánh nhầm điểm. Thứ ba, khó khăn trong lưu trữ và thống kê: các phiếu thi giấy chiếm nhiều không gian lưu trữ, việc thống kê phân tích kết quả thi phải thực hiện thủ công bằng bảng tính.

Trên thị trường ứng dụng, một số sản phẩm đáng chú ý bao gồm: ứng dụng chấm thi trắc nghiệm của các công ty EdTech Việt Nam như Viettel, FPT; các ứng dụng đơn giản trên điện thoại chỉ hỗ trợ đếm điểm đơn giản; và các phần mềm OMR trên máy tính cần kết hợp máy scan. Phần lớn các ứng dụng hiện tại mới chỉ dừng ở chức năng nhận diện và tính điểm, chưa tích hợp phân tích dữ liệu thông minh bằng AI.

Bảng 2.1 dưới đây trình bày kết quả khảo sát và so sánh một số sản phẩm tương

tự.

Tiêu chí	Sản phẩm A (phần mềm OMR tính)	Sản phẩm B (ứng dụng di động)	Sản phẩm C (EdTech)	SMART GRADING
Nền tảng	Máy tính + máy scan	Di động	Web + Di động	Di động
Quét OMR	Có	Hạn chế	Không	Có
OCR nhận dạng	Không	Có	Không	Có
Phân tích AI	Không	Không	Hạn chế	Có
Chi phí	Cao (thiết bị scan)	Thấp	Trung bình	Thấp (chỉ cần điện thoại)
Độ chính xác	Cao	Trung bình	Không áp dụng	Cao
Phân tích phổ điểm	Thủ công	Không	Hạn chế	Tự động bằng AI

Table 2.1: So sánh các sản phẩm chấm thi trắc nghiệm trên thị trường

Qua khảo sát, hạn chế chung của các sản phẩm hiện tại là chưa kết hợp được khả năng quét OMR di động với phân tích thông minh bằng Generative AI trên cùng một ứng dụng, đây chính là khoảng trống mà đề tài SMART GRADING hướng tới lấp đầy.

2.3 Tổng quan chức năng

Phần 2.3 này tóm tắt các chức năng tổng quan của hệ thống SMART GRADING ở mức cao. Hệ thống được thiết kế với hai nhóm người dùng chính: giáo viên và học sinh. Giáo viên có thể thực hiện các thao tác quản lý lớp học, quản lý đề thi, quét phiếu trả lời, xem kết quả chấm thi và phân tích chi tiết bằng AI. Học sinh có thể xem kết quả thi cá nhân và nộp đơn phúc khảo nếu không đồng ý với kết quả.

2.3.1 Biểu đồ use case tổng quát

Hệ thống SMART GRADING có ba tác nhân chính tham gia: Giáo viên, Học sinh và Hệ thống (bao gồm các module backend, OCR và AI). Biểu đồ use case tổng quát được mô tả trong Hình 2.1.

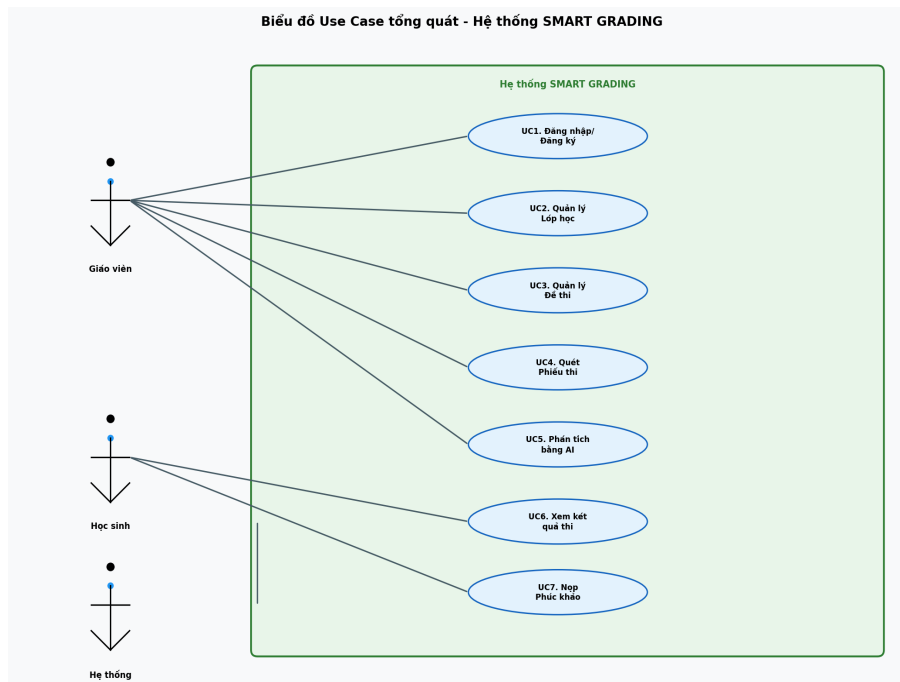


Figure 2.1: Biểu đồ use case tổng quát của hệ thống SMART GRADING

Các use case chính của hệ thống được mô tả như sau. Use case Đăng nhập/Đăng ký cho phép người dùng xác thực danh tính để truy cập hệ thống. Use case Quản lý lớp học cho phép giáo viên tạo mới, chỉnh sửa, xóa lớp học và thêm học sinh vào lớp. Use case Quản lý đề thi cho phép giáo viên tạo, chỉnh sửa, xóa và xuất bản đề thi. Use case Quét phiếu thi là chức năng cốt lõi, cho phép giáo viên sử dụng camera để chụp ảnh phiếu trả lời trắc nghiệm, hệ thống tự động nhận diện vùng đáp án, trích xuất kết quả và chấm điểm. Use case Phân tích kết quả bằng AI cho phép giáo viên yêu cầu Generative AI phân tích kết quả bài thi và nhận về các nhận xét chi tiết. Use case Xem kết quả thi cho phép giáo viên và học sinh xem điểm số và chi tiết từng câu. Use case Nộp phúc khảo cho phép học sinh gửi yêu cầu phúc khảo cho một câu hỏi cụ thể.

2.3.2 Biểu đồ use case phân rã Quét phiếu thi

Use case Quét phiếu thi là chức năng cốt lõi nhất của hệ thống. Biểu đồ use case phân rã được thể hiện trong Hình 2.2.

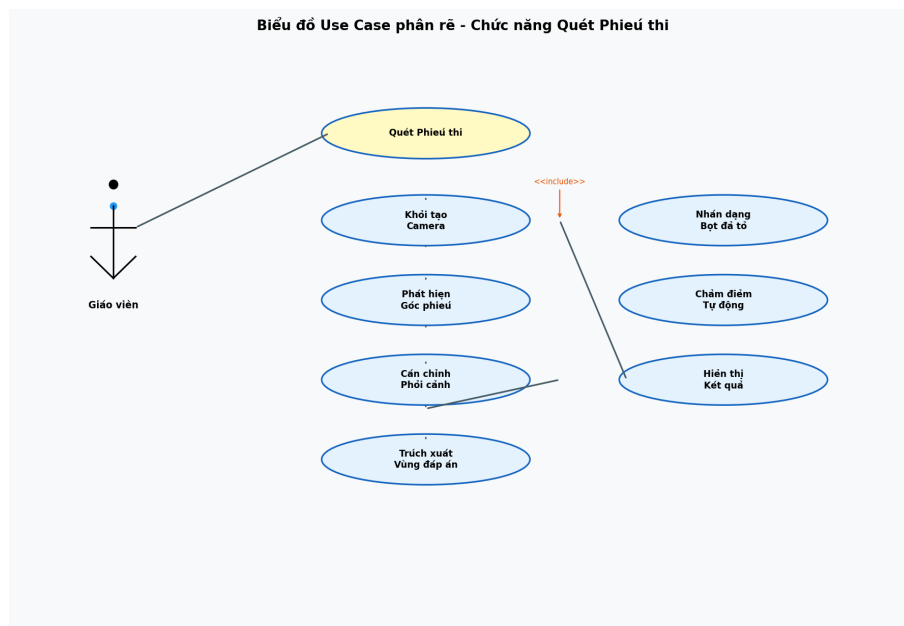


Figure 2.2: Biểu đồ use case phân rã chức năng Quét phiếu thi

Use case Quét phiếu thi được phân rã thành các use case con: Khởi tạo camera để chuẩn bị chế độ quét; Phát hiện góc phiếu để nhận diện bốn góc của phiếu thi trong khung hình; Căn chỉnh phối cảnh để biến đổi ảnh chụp lệch thành ảnh phẳng; Trích xuất vùng đáp án để tách riêng vùng chứa các bọt trả lời; Nhận dạng bọt đã tô bằng kỹ thuật OCR/OMR để xác định đáp án được chọn; Chấm điểm tự động bằng cách đối chiếu với đáp án mẫu; và Hiển thị kết quả để trình bày điểm số cùng các chi tiết.

2.3.3 Quy trình nghiệp vụ

Quy trình nghiệp vụ chính của hệ thống SMART GRADING bao gồm ba giai đoạn: chuẩn bị, thực hiện thi và phân tích kết quả. Giai đoạn chuẩn bị bắt đầu khi giáo viên đăng nhập hệ thống, tạo hoặc chọn lớp học, tạo đề thi với danh sách câu hỏi và đáp án, thiết lập thông tin phiên thi. Giai đoạn thực hiện thi bắt đầu khi giáo viên chọn chức năng quét, mở camera và chụp ảnh từng phiếu trả lời của học sinh, hệ thống tự động xử lý ảnh và hiển thị kết quả chấm điểm, giáo viên xác nhận và lưu kết quả. Giai đoạn phân tích bắt đầu khi giáo viên chọn phiên thi đã chấm xong, gửi yêu cầu phân tích AI, hệ thống tổng hợp dữ liệu điểm số và gửi tới API Generative AI, nhận về báo cáo phân tích và hiển thị cho giáo viên.

2.4 Đặc tả chức năng

Phần này đặc tả chi tiết bốn use case quan trọng nhất của hệ thống: Quản lý đề thi, Quét phiếu thi, Phân tích kết quả bằng AI và Xem kết quả thi.

2.4.1 Đặc tả use case Quản lý đề thi

Tên use case: Quản lý đề thi

Tác nhân chính: Giáo viên

Mô tả: Cho phép giáo viên tạo mới, chỉnh sửa, xóa và xuất bản đề thi trắc nghiệm.

Tiền điều kiện: Giáo viên đã đăng nhập thành công vào hệ thống.

Luồng sự kiện chính:

1. Giáo viên chọn chức năng Quản lý đề thi từ màn hình chính.
2. Hệ thống hiển thị danh sách các đề thi đã tạo, bao gồm tiêu đề, số câu hỏi, ngày tạo và trạng thái (bản nháp hoặc đã xuất bản).
3. Giáo viên chọn tạo đề thi mới.
4. Hệ thống hiển thị biểu mẫu tạo đề thi, yêu cầu nhập tiêu đề đề thi, chọn lớp học, nhập danh sách câu hỏi với các phương án A, B, C, D và đáp án đúng.
5. Giáo viên nhập thông tin và bấm lưu.
6. Hệ thống kiểm tra dữ liệu đầu vào, lưu vào cơ sở dữ liệu và hiển thị thông báo thành công.
7. Giáo viên có thể chọn xuất bản đề thi để kích hoạt để sử dụng trong phiên chấm thi.

Luồng sự kiện phát sinh - Chỉnh sửa đề thi:

- a) Giáo viên chọn đề thi từ danh sách và bấm chỉnh sửa.
- b) Hệ thống hiển thị biểu mẫu với thông tin hiện tại của đề thi.
- c) Giáo viên thay đổi thông tin và bấm lưu, hệ thống cập nhật vào cơ sở dữ liệu.

Luồng sự kiện phát sinh - Xóa đề thi:

- a) Giáo viên chọn đề thi và bấm xóa.
- b) Hệ thống hiển thị hộp thoại xác nhận xóa.
- c) Giáo viên xác nhận xóa, hệ thống xóa đề thi khỏi cơ sở dữ liệu.

Hậu điều kiện: Đề thi được lưu thành công vào cơ sở dữ liệu và có thể được sử dụng trong phiên chấm thi.

2.4.2 Đặc tả use case Quét phiếu thi

Tên use case: Quét phiếu thi

Tác nhân chính: Giáo viên

Mô tả: Cho phép giáo viên sử dụng camera thiết bị di động để chụp ảnh phiếu trả lời trắc nghiệm, hệ thống tự động nhận diện góc phiếu, căn chỉnh phối cảnh, trích xuất đáp án và chấm điểm.

Tiền điều kiện: Giáo viên đã đăng nhập và đã có ít nhất một đề thi đã xuất bản.

Luồng sự kiện chính:

1. Giáo viên chọn chức năng Quét phiếu thi từ màn hình chính.
2. Hệ thống hiển thị giao diện camera với khung hướng dẫn đặt phiếu thi.
3. Giáo viên đặt phiếu thi trắc nghiệm trước camera, hệ thống tự động phát hiện bốn góc phiếu và hiển thị đường viền phát hiện.
4. Giáo viên bấm nút chụp, hệ thống chụp ảnh và thực hiện căn chỉnh phối cảnh.
5. Hệ thống trích xuất vùng chứa các bọt đáp án và áp dụng kỹ thuật OCR/OMR để nhận dạng các bọt đã tô.
6. Hệ thống đối chiếu đáp án nhận diện được với đáp án mẫu của đề thi đã chọn và tính điểm.
7. Hệ thống hiển thị kết quả chấm điểm chi tiết: tổng điểm, số câu đúng, số câu sai, danh sách các câu sai kèm đáp án đúng.
8. Giáo viên xác nhận kết quả hoặc điều chỉnh thủ công một số câu nếu cần.
9. Hệ thống lưu kết quả chấm thi vào cơ sở dữ liệu.

Luồng sự kiện phát sinh - Điều chỉnh thủ công:

- a) Tại bước 7, giáo viên bấm nút sửa để chỉnh sửa thủ công một số câu.
- b) Hệ thống hiển thị danh sách các câu với phương án A, B, C, D.
- c) Giáo viên thay đổi đáp án và bấm xác nhận, hệ thống tính lại điểm.

Luồng sự kiện phát sinh - Không phát hiện được góc phiếu:

- a) Tại bước 3, nếu hệ thống không phát hiện được góc phiếu sau 5 giây, hiển thị thông báo hướng dẫn điều chỉnh góc chụp.
- b) Giáo viên điều chỉnh vị trí phiếu và góc camera, hệ thống tiếp tục phát hiện.

Hậu điều kiện: Kết quả chấm thi được lưu thành công, kết quả được hiển thị cho giáo viên và có thể xem lại sau đó.

2.4.3 Đặc tả use case Phân tích kết quả bằng AI

Tên use case: Phân tích kết quả bằng AI

Tác nhân chính: Giáo viên, Hệ thống (Generative AI)

Mô tả: Cho phép giáo viên yêu cầu hệ thống phân tích kết quả bài thi của toàn lớp bằng Generative AI, nhận về báo cáo tổng hợp phổ điểm, các câu hỏi học sinh hay sai và nhận xét chi tiết.

Tiền điều kiện: Giáo viên đã hoàn thành chấm thi cho một phiên thi với ít nhất 5 phiếu.

Luồng sự kiện chính:

1. Giáo viên chọn phiên thi đã chấm xong từ danh sách kết quả.
2. Giáo viên bấm nút Phân tích bằng AI.
3. Hệ thống tổng hợp toàn bộ dữ liệu điểm số, danh sách câu đúng sai của từng học sinh.
4. Hệ thống gửi dữ liệu tổng hợp tới API Generative AI kèm prompt định sẵn yêu cầu phân tích.
5. Generative AI trả về báo cáo phân tích bao gồm: phổ điểm, tỷ lệ đạt yêu cầu, danh sách các câu hỏi có tỷ lệ sai cao nhất, nguyên nhân phổ biến dẫn đến sai và đề xuất phương pháp cải thiện.
6. Hệ thống hiển thị báo cáo phân tích cho giáo viên dưới dạng văn bản và biểu đồ.
7. Giáo viên có thể lưu báo cáo hoặc chia sẻ với đồng nghiệp.

Hậu điều kiện: Báo cáo phân tích AI được hiển thị cho giáo viên và có thể lưu trữ để tham khảo.

2.4.4 Đặc tả use case Xem kết quả thi

Tên use case: Xem kết quả thi

Tác nhân chính: Giáo viên, Học sinh

Mô tả: Cho phép người dùng xem chi tiết kết quả bài thi, bao gồm điểm số, số câu đúng sai và danh sách các câu đã làm.

Tiền điều kiện: Người dùng đã đăng nhập và có kết quả thi trong hệ thống.

Luồng sự kiện chính:

1. Người dùng chọn chức năng Xem kết quả thi.
2. Hệ thống hiển thị danh sách các bài thi đã tham gia với thông tin cơ bản: tên đề thi, ngày thi, điểm số.
3. Người dùng chọn một bài thi cụ thể.
4. Hệ thống hiển thị chi tiết: tổng điểm, biểu đồ phổ điểm của lớp (nếu là giáo viên), danh sách từng câu với trạng thái đúng/sai.
5. Với mỗi câu sai, hệ thống hiển thị đáp án đúng và phương án học sinh đã chọn.

Luồng sự kiện phát sinh - Nộp phúc khảo (Học sinh):

- a) Học sinh chọn một câu hỏi cụ thể mà em cho là bị chấm sai.
- b) Hệ thống hiển thị biểu mẫu yêu cầu phúc khảo, cho phép học sinh nhập lý do.
- c) Học sinh gửi yêu cầu phúc khảo, giáo viên nhận được thông báo và xem xét.

Hậu điều kiện: Người dùng xem được chi tiết kết quả bài thi.

2.5 Yêu cầu phi chức năng

Bên cạnh các yêu cầu chức năng đã đặc tả ở trên, hệ thống SMART GRADING cần đáp ứng các yêu cầu phi chức năng sau đây.

Về hiệu năng, thời gian xử lý một ảnh phiếu thi từ lúc chụp đến khi hiển thị kết quả không quá 3 giây trên thiết bị có cấu hình trung bình. Thời gian phản hồi của API Generative AI không quá 10 giây cho mỗi yêu cầu phân tích. Hệ thống hỗ trợ đồng thời tối thiểu 50 người dùng truy cập cùng lúc.

Về độ tin cậy, tỷ lệ nhận dạng đáp án đúng của module OCR/OMR đạt tối thiểu 95% trong điều kiện ánh sáng bình thường và phiếu thi không bị nhàu nát nghiêm trọng. Hệ thống cần lưu trữ kết quả thi một cách an toàn, đảm bảo dữ liệu không bị mất khi có sự cố.

Về tính dễ dùng, giao diện người dùng cần trực quan, dễ hiểu để giáo viên ở mọi lứa tuổi có thể sử dụng được mà không cần đào tạo nhiều. Quy trình quét phiếu thi được thiết kế đơn giản, tối đa 3 thao tác từ khi mở camera đến khi có kết quả.

Về tính dễ bảo trì, hệ thống được thiết kế theo kiến trúc module hóa, cho phép dễ dàng nâng cấp và mở rộng. Mã nguồn được tổ chức rõ ràng, có tài liệu mô tả API và cấu trúc dữ liệu.

Về yêu cầu kỹ thuật, ứng dụng di động hỗ trợ Android phiên bản 6.0 (API 23)

trở lên và iOS phiên bản 12.0 trở lên. Backend sử dụng Node.js với cơ sở dữ liệu MongoDB. Module OCR sử dụng OpenCV cho xử lý ảnh và Google ML Kit hoặc Tesseract cho nhận dạng ký tự.

Về bảo mật, toàn bộ dữ liệu truyền tải giữa ứng dụng và server sử dụng giao thức HTTPS. Thông tin đăng nhập và token xác thực được mã hóa và lưu trữ an toàn trên thiết bị. Phân quyền người dùng được kiểm soát chặt chẽ ở cả tầng ứng dụng và tầng backend.

CHƯƠNG 3. NỀN TẢNG LÝ THUYẾT VÀ CÔNG NGHỆ SỬ DỤNG

3.1 Tổng quan

Chương 2 đã trình bày hiện trạng quy trình chấm thi trắc nghiệm, phân tích các sản phẩm tương tự và đặc tả chi tiết các yêu cầu chức năng cùng phi chức năng của hệ thống SMART GRADING. Chương 3 này sẽ giới thiệu các nền tảng lý thuyết và công nghệ được lựa chọn để xây dựng hệ thống, bao gồm Flutter cho phát triển ứng dụng di động, các kỹ thuật xử lý ảnh và thị giác máy tính (OpenCV), kỹ thuật Nhận dạng ký tự quang học (OCR), cơ sở dữ liệu MongoDB, Node.js cho backend, và Generative AI cùng cách tích hợp API. Mỗi phần sẽ trình bày lý thuyết nền tảng, các công nghệ và framework liên quan, đồng thời phân tích các lựa chọn thay thế và lý do lựa chọn công nghệ cụ thể.

3.2 Nền tảng phát triển ứng dụng di động

3.2.1 Lựa chọn Flutter

Trong quá trình nghiên cứu để lựa chọn nền tảng phát triển ứng dụng di động cho hệ thống SMART GRADING, có ba phương án chính được xem xét: phát triển ứng dụng native trên Android bằng Kotlin hoặc trên iOS bằng Swift, sử dụng framework cross-platform như Flutter hoặc React Native, hoặc phát triển ứng dụng web di động (Progressive Web App). Phát triển native cho phép tận dụng tối đa khả năng phần cứng thiết bị, đặc biệt quan trọng với các tác vụ xử lý ảnh thời gian thực, tuy nhiên chi phí phát triển cao gấp đôi do phải viết và duy trì hai codebase riêng biệt cho Android và iOS.

Flutter được lựa chọn làm framework phát triển chính cho hệ thống SMART GRADING vì nhiều lý do thuyết phục. Flutter sử dụng ngôn ngữ Dart, biên dịch trực tiếp sang mã máy (ARM code) thông qua công nghệ AOT (Ahead-of-Time) compilation, cho hiệu năng gần như native và khả năng phản hồi nhanh, đặc biệt phù hợp với các tác vụ xử lý ảnh thời gian thực từ camera. Kiến trúc rendering của Flutter sử dụng engine Skia vẽ trực tiếp lên canvas, không phụ thuộc vào các widget native của hệ điều hành, đảm bảo tính nhất quán của giao diện trên mọi thiết bị. Hệ sinh thái plugin pub.dev cung cấp rất nhiều gói mở rộng cho camera, xử lý ảnh, HTTP request và các chức năng khác. Việc xây dựng một codebase duy nhất chạy trên cả Android và iOS giúp tiết kiệm thời gian và chi phí phát triển so với việc duy trì hai codebase riêng biệt.

Flutter sử dụng kiến trúc BLoC (Business Logic Component) để quản lý trạng thái ứng dụng, trong đó mỗi feature được chia thành ba thành phần chính: Events

(sự kiện từ giao diện người dùng), States (trạng thái của feature tại một thời điểm) và BLoC (xử lý logic nghiệp vụ, nhận events và phát ra states mới). Kiến trúc này đảm bảo sự tách biệt rõ ràng giữa logic nghiệp vụ và giao diện, giúp mã nguồn dễ đọc, dễ kiểm thử và dễ bảo trì.

3.2.2 Các công cụ và thư viện hỗ trợ trên Flutter

Để xây dựng hệ thống SMART GRADING trên nền tảng Flutter, một số thư viện quan trọng được sử dụng như sau. Thư viện flutter_bloc (phiên bản 9.1.1) được sử dụng để cài đặt kiến trúc BLoC, quản lý trạng thái ứng dụng theo mô hình reactive. Thư viện camera và image_picker được sử dụng để truy cập camera thiết bị, chụp ảnh và chọn ảnh từ thư viện. Thư viện dio được sử dụng làm HTTP client để giao tiếp với backend API. Thư viện opencv_dart cho phép tích hợp OpenCV vào Flutter, phục vụ các tác vụ xử lý ảnh nâng cao như phát hiện cạnh, biến đổi phối cảnh và tách vùng. Thư viện flutter_secure_storage được sử dụng để lưu trữ token xác thực một cách an toàn trên thiết bị.

3.3 Xử lý ảnh và Thị giác máy tính

3.3.1 Khái quát về xử lý ảnh trên di động

Xử lý ảnh trên thiết bị di động đặt ra nhiều thách thức đặc thù so với xử lý trên máy tính. Thiết bị di động có tài nguyên tính toán hạn chế, đặc biệt khi xử lý ảnh độ phân giải cao cần nhiều bộ nhớ và CPU. Ngoài ra, nguồn pin hạn chế đòi hỏi thuật toán phải tối ưu về mặt năng lượng. Camera di động chất lượng thấp hơn camera chuyên dụng, ảnh chụp thường bị nhiễu, méo, thiếu sáng hoặc dư sáng. Tốc độ xử lý cần đủ nhanh để đảm bảo trải nghiệm người dùng mượt mà.

OpenCV (Open Source Computer Vision Library) là thư viện mã nguồn mở chuẩn công nghiệp cho xử lý ảnh và thị giác máy tính. Phiên bản OpenCV 4.x cung cấp hàng trăm thuật toán từ cơ bản đến nâng cao, bao gồm các thao tác xử lý ảnh cơ bản như lọc nhiễu, phát hiện cạnh, biến đổi hình học, cho tới các thuật toán phức tạp như phát hiện đặc trưng, matching hình ảnh. OpenCV được viết bằng C++ với bindings cho Python, Java và các ngôn ngữ khác. Trên nền tảng di động, OpenCV hỗ trợ Android và iOS, có thể tích hợp qua CMake hoặc thông qua các package như opencv4nodejs hoặc opencv_dart.

Một số lựa chọn thay thế cho xử lý ảnh trên di động bao gồm sử dụng các API cloud-based như Google Cloud Vision API hoặc Amazon Rekognition, cho phép xử lý ảnh trên server mạnh và độ chính xác cao, tuy nhiên yêu cầu kết nối internet liên tục và phát sinh chi phí theo số lượng request. TensorFlow Lite cho phép triển khai các mô hình machine learning trên di động, phù hợp cho nhận diện đối tượng phức tạp, nhưng đòi hỏi huấn luyện mô hình riêng cho bài toán OMR và tốn nhiều

công sức. Giải pháp được lựa chọn cho hệ thống SMART GRADING là sử dụng kết hợp OpenCV cho xử lý ảnh cơ bản và thuật toán nhận dạng bọt tự xây dựng, vì yêu cầu xử lý theo thời gian thực, không phụ thuộc internet và cần kiểm soát chi phí.

3.3.2 Các kỹ thuật xử lý ảnh cốt lõi cho bài toán OMR

Bài toán nhận dạng phiếu trắc nghiệm (OMR) đòi hỏi một chuỗi các kỹ thuật xử lý ảnh được thực hiện tuần tự. Mỗi kỹ thuật đóng vai trò quan trọng trong việc nâng cao độ chính xác nhận dạng.

Phát hiện góc tài liệu (Document Corner Detection) là bước đầu tiên và quan trọng nhất. Mục tiêu là xác định chính xác vị trí bốn góc của phiếu thi trong ảnh chụp từ camera. Thuật toán phổ biến bao gồm sử dụng Hough Transform để phát hiện các đường thẳng và giao điểm của chúng làm góc, hoặc sử dụng phát hiện contour và xấp xỉ đa giác để tìm hình chữ nhật có bốn đỉnh. Trong hệ thống SMART GRADING, phương pháp contour-based detection kết hợp với Douglas-Peucker algorithm để xấp xỉ đa giác được sử dụng, vì cho kết quả tốt với phiếu thi có viền rõ ràng.

Căn chỉnh phối cảnh (Perspective Transformation) là bước tiếp theo, nhằm biến đổi ảnh chụp từ góc nghiêng thành ảnh nhìn thẳng, đúng tỷ lệ. Kỹ thuật này sử dụng phép biến đổi perspective transform dựa trên bốn điểm góc đã xác định ở bước trước. Ma trận biến đổi được tính toán bằng hàm `getPerspectiveTransform` của OpenCV, sau đó áp dụng `warpPerspective` để tạo ảnh đã căn chỉnh.

Tiền xử lý ảnh (Image Preprocessing) bao gồm các bước nâng cao chất lượng ảnh trước khi nhận dạng. Các kỹ thuật được áp dụng bao gồm chuyển đổi sang ảnh xám (grayscale) để giảm độ phức tạp tính toán, áp dụng Gaussian blur để giảm nhiễu, sử dụng adaptive threshold (phương pháp Otsu hoặc Adaptive Gaussian) để nhị phân hóa ảnh, tách riêng vùng trắng và vùng đen một cách rõ ràng.

Tách vùng bọt đáp án (Bubble Segmentation) là bước phân đoạn ảnh để xác định chính xác vị trí từng bọt trả lời trên phiếu. Dựa trên template phiếu thi đã biết trước (khoảng cách, kích thước các bọt), hệ thống sử dụng phương pháp projection profile analysis để xác định ranh giới các hàng và cột chứa bọt. Mỗi bọt được cắt riêng thành vùng ảnh con để phân tích mức độ tô đậm.

Nhận dạng bọt đã tô (Bubble Fill Detection) là bước quan trọng nhất quyết định độ chính xác của toàn bộ hệ thống. Phương pháp được sử dụng là tính toán tổng giá trị pixel trong vùng bọt: nếu tổng này vượt qua một ngưỡng (threshold) nhất định, bọt đó được coi là đã được tô. Ngưỡng này có thể là ngưỡng cố định hoặc

ngưỡng động được tính dựa trên phân bố giá trị pixel của tất cả các bọt trong cùng một hàng, giúp thích ứng với điều kiện ánh sáng khác nhau.

3.4 Kỹ thuật Nhận dạng ký tự quang học (OCR)

Nhận dạng ký tự quang học (Optical Character Recognition - OCR) là kỹ thuật chuyển đổi hình ảnh chứa văn bản thành văn bản số mà máy tính có thể đọc và xử lý. Trong bài toán OMR, OCR không chỉ đơn thuần nhận dạng ký tự mà còn phải nhận dạng vùng tô đậm trên phiếu thi.

Google ML Kit là bộ công cụ machine learning của Google dành cho các ứng dụng di động, cung cấp các mô hình ML đã được huấn luyện sẵn và tối ưu cho thiết bị di động. ML Kit bao gồm module Text Recognition hỗ trợ nhận dạng văn bản Latin, có thể sử dụng để nhận dạng các ký hiệu, mã phiếu in trên phiếu thi. ML Kit chạy trên thiết bị (on-device), không cần kết nối internet, và được tối ưu hóa cho tốc độ và pin.

Tesseract là engine OCR mã nguồn mở phổ biến nhất hiện nay, được phát triển ban đầu bởi HP Labs và sau đó được Google bảo trì. Tesseract hỗ trợ hơn 100 ngôn ngữ và có khả năng nhận dạng cao với các hình ảnh chất lượng tốt. Tuy nhiên, Tesseract yêu cầu xử lý trên máy tính hoặc server, không chạy trực tiếp trên thiết bị di động mà cần thông qua server, gây phụ thuộc vào kết nối mạng.

Đối với hệ thống SMART GRADING, do bài toán OMR chủ yếu là nhận dạng vùng tô (mark detection) chứ không phải nhận dạng ký tự phức tạp, phương pháp tiếp cận kết hợp giữa OpenCV và thuật toán fill-intensity analysis được lựa chọn thay vì dùng OCR engine nặng. Kỹ thuật này dựa trên nguyên lý: một bọt trắng trên giấy khi được tô đậm bằng bút chì hoặc bút bi sẽ có mật độ pixel tối (pixel intensity thấp) khác biệt rõ ràng so với bọt không được tô. Bằng cách tính toán giá trị trung bình pixel trong vùng mỗi bọt và so sánh với ngưỡng, hệ thống xác định bọt nào đã được chọn với độ chính xác cao.

3.5 Cơ sở dữ liệu và Backend

Hệ thống SMART GRADING sử dụng MongoDB làm cơ sở dữ liệu chính. MongoDB là cơ sở dữ liệu NoSQL document-oriented, lưu trữ dữ liệu dưới dạng JSON-like documents (BSON), cho phép linh hoạt trong việc thay đổi cấu trúc dữ liệu mà không cần migrate schema phức tạp. Điều này đặc biệt phù hợp với bài toán quản lý đề thi và câu hỏi, nơi mà cấu trúc một câu hỏi có thể thay đổi theo thời gian (ví dụ: thêm trường hình ảnh, thêm giải thích). MongoDB cũng có khả năng mở rộng theo chiều ngang (horizontal scaling) tốt, phù hợp với mô hình phát triển ban đầu và có thể mở rộng sau này.

Node.js với Express.js được lựa chọn làm runtime và framework backend. Node.js sử dụng mô hình I/O bất đồng bộ (non-blocking I/O) xử lý đồng thời nhiều request hiệu quả, đặc biệt phù hợp với các ứng dụng real-time và I/O-heavy như ứng dụng chat, streaming. Express.js là framework web nhẹ, linh hoạt và dễ sử dụng cho Node.js, cung cấp hệ thống middleware mạnh mẽ cho việc xử lý request, authentication và validation. Hệ sinh thái npm của Node.js cung cấp hàng trăm nghìn gói thư viện hữu ích cho mọi nhu cầu.

Một số lựa chọn thay thế bao gồm Django (Python) với ORM mạnh mẽ và admin interface sẵn có, phù hợp với các dự án cần xây dựng nhanh nhưng kiến trúc đơn giản hơn và ít real-time features. Spring Boot (Java) là lựa chọn mạnh mẽ cho các hệ thống doanh nghiệp với yêu cầu bảo mật và scalability cao, tuy nhiên độ phức tạp cài đặt và đường cong học tập cao hơn. Go (Golang) cho hiệu năng cao và concurrency xuất sắc, phù hợp với các hệ thống cần xử lý request lớn, nhưng hệ sinh thái thư viện cho web development chưa phong phú bằng Node.js.

3.6 Generative AI và tích hợp API

Generative AI (GenAI) là nhánh của trí tuệ nhân tạo có khả năng tạo ra nội dung mới bao gồm văn bản, hình ảnh, âm nhạc hoặc mã nguồn, dựa trên việc học từ dữ liệu huấn luyện. Các mô hình GenAI tiền đào tạo (pretrained) như GPT-4, Claude, Gemini đã đạt được khả năng hiểu ngôn ngữ tự nhiên ở mức gần con người, cho phép tích hợp vào các ứng dụng thông qua API một cách dễ dàng.

Trong hệ thống SMART GRADING, Generative AI được tích hợp để thực hiện các tác vụ phân tích dữ liệu thông minh. Thay vì chỉ trả về điểm số đơn thuần, hệ thống gửi dữ liệu kết quả thi tổng hợp tới API Generative AI và nhận về báo cáo phân tích chi tiết bao gồm phổ điểm, xu hướng, các câu hỏi khó và gợi ý cải thiện. Việc tích hợp được thực hiện thông qua HTTP request tới API endpoint của nhà cung cấp GenAI, sử dụng prompt engineering để định dạng yêu cầu và xử lý response.

Một số lựa chọn API GenAI bao gồm OpenAI GPT-4 API cung cấp khả năng ngôn ngữ xuất sắc, document processing mạnh mẽ, tuy nhiên chi phí sử dụng cao hơn và cần tuân thủ các quy định về dữ liệu. Google Gemini API là mô hình đa phương thức (multimodal) hỗ trợ text, image, audio trong cùng một model, chi phí cạnh tranh hơn. Anthropic Claude cung cấp khả năng phân tích chi tiết, long context window và safety alignment tốt. Hệ thống SMART GRADING sử dụng Google Gemini API vì hỗ trợ multimodal, chi phí hợp lý, dễ tích hợp và có tier miễn phí cho mục đích phát triển.

3.7 Kết chương

Chương này đã trình bày chi tiết các nền tảng lý thuyết và công nghệ được lựa chọn để xây dựng hệ thống SMART GRADING. Flutter được chọn làm framework phát triển ứng dụng di động đa nền tảng với kiến trúc BLoC, mang lại hiệu năng cao và trải nghiệm người dùng nhất quán trên Android và iOS. Kỹ thuật xử lý ảnh với OpenCV kết hợp thuật toán fill-intensity analysis được áp dụng cho bài toán nhận dạng bọ đáp án, không phụ thuộc internet và kiểm soát chi phí. MongoDB và Node.js/Express cung cấp nền tảng backend linh hoạt, dễ mở rộng. Generative AI (Google Gemini) được tích hợp để mang lại giá trị phân tích thông minh vượt trội so với các giải pháp chấm thi thông thường. Từ các công nghệ đã chọn, chương tiếp theo sẽ trình bày chi tiết về phân tích thiết kế và triển khai hệ thống SMART GRADING.

CHƯƠNG 4. PHÂN TÍCH THIẾT KẾ, TRIỂN KHAI VÀ ĐÁNH GIÁ HỆ THỐNG

4.1 Tổng quan

Chương 3 đã giới thiệu các nền tảng lý thuyết và công nghệ được sử dụng trong hệ thống SMART GRADING. Chương 4 này trình bày chi tiết về phân tích và thiết kế hệ thống, bao gồm lựa chọn kiến trúc phần mềm, thiết kế tổng quan, thiết kế chi tiết các module, thiết kế cơ sở dữ liệu, thiết kế giao diện người dùng, kết quả xây dựng ứng dụng, quá trình kiểm thử và triển khai thử nghiệm.

4.2 Thiết kế kiến trúc

4.2.1 Lựa chọn kiến trúc phần mềm

Hệ thống SMART GRADING được thiết kế theo kiến trúc đa tầng (Multi-tier Architecture) kết hợp với mô hình BLoC (Business Logic Component) cho tầng xử lý nghiệp vụ trên ứng dụng di động. Kiến trúc đa tầng là mô hình trong đó hệ thống được chia thành nhiều tầng (layers) rõ ràng, mỗi tầng có trách nhiệm riêng biệt và chỉ giao tiếp với tầng ngay bên dưới hoặc bên trên nó.

Cụ thể, ứng dụng di động SMART GRADING được tổ chức thành ba tầng chính. Tầng Presentation (Trình bày) chứa các widget giao diện người dùng (UI widgets) và các BLoC xử lý trạng thái, chịu trách nhiệm hiển thị dữ liệu và tương tác với người dùng. Tầng Domain (Nghiệp vụ) chứa các entity (mô hình dữ liệu nghiệp vụ) và repository interfaces, định nghĩa các nghiệp vụ cốt lõi mà hệ thống thực hiện. Tầng Data (Dữ liệu) chứa các repository implementation, API services và local storage, chịu trách nhiệm lấy dữ liệu từ backend hoặc lưu trữ cục bộ.

Kiến trúc BLoC được lựa chọn cho tầng Presentation vì nhiều ưu điểm vượt trội. Mô hình reactive đảm bảo giao diện luôn đồng bộ với trạng thái nghiệp vụ. Luồng dữ liệu một chiều (unidirectional data flow) giúp code dễ hiểu và debug: Event đi vào BLoC, BLoC xử lý và phát ra State mới, UI nhận State và render lại. BLoC tách biệt hoàn toàn logic nghiệp vụ khỏi UI, cho phép kiểm thử BLoC một cách độc lập mà không cần mock widget. Kiến trúc này cũng hỗ trợ tốt cho việc tái sử dụng code và làm việc nhóm.

Backend của hệ thống SMART GRADING tuân theo mô hình ba tầng MVC truyền thống. Tầng Controller nhận request từ client, gọi tầng Service tương ứng và trả response về. Tầng Service chứa logic nghiệp vụ chính, xử lý validation, tính toán và điều phối giữa các model. Tầng Model chứa schema định nghĩa cấu trúc dữ liệu trong MongoDB và các phương thức truy vấn.

Backend được triển khai dưới dạng API RESTful, trong đó mỗi endpoint đại diện cho một tài nguyên (resource) và một phương thức HTTP tương ứng với một thao tác CRUD. Ví dụ: GET /api/v1/exams để lấy danh sách đề thi, POST /api/v1/exams để tạo đề thi mới, PUT /api/v1/exams/:id để cập nhật, DELETE /api/v1/exams/:id để xóa.

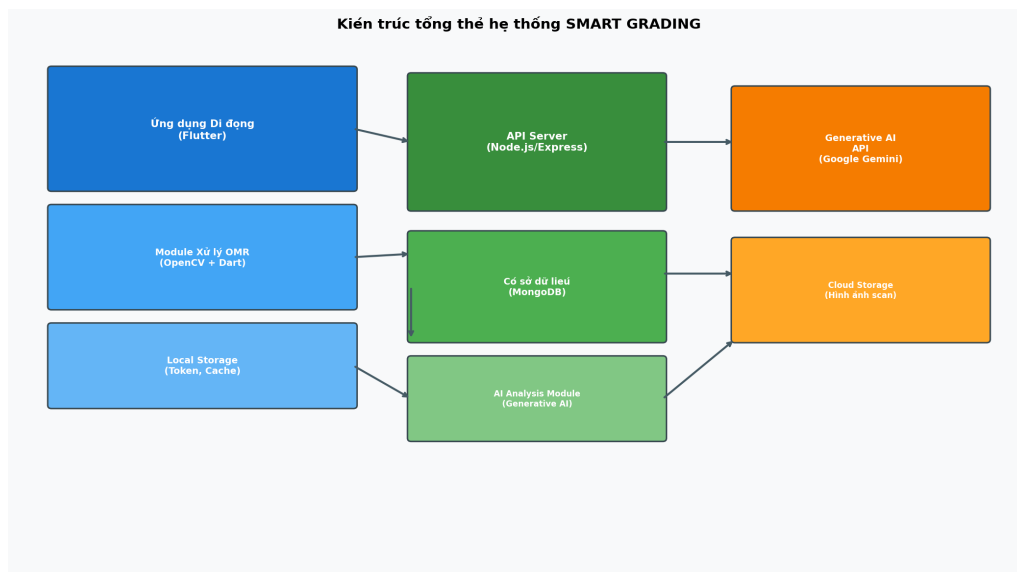


Figure 4.1: Sơ đồ kiến trúc tổng thể hệ thống SMART GRADING

Hình 4.1 mô tả kiến trúc tổng thể của hệ thống SMART GRADING. Ứng dụng di động Flutter giao tiếp với Backend Node.js/Express thông qua API REST. Backend kết nối với MongoDB để lưu trữ dữ liệu và gọi API Generative AI (Google Gemini) để phân tích kết quả thi. Module xử lý OMR trên ứng dụng di động xử lý ảnh cục bộ, không cần server, đảm bảo tốc độ và bảo mật.

4.2.2 Thiết kế tổng quan

Hệ thống SMART GRADING được tổ chức thành các gói (package) theo cấu trúc phân tầng rõ ràng. Hình 4.2 thể hiện biểu đồ các gói chính của ứng dụng di động.

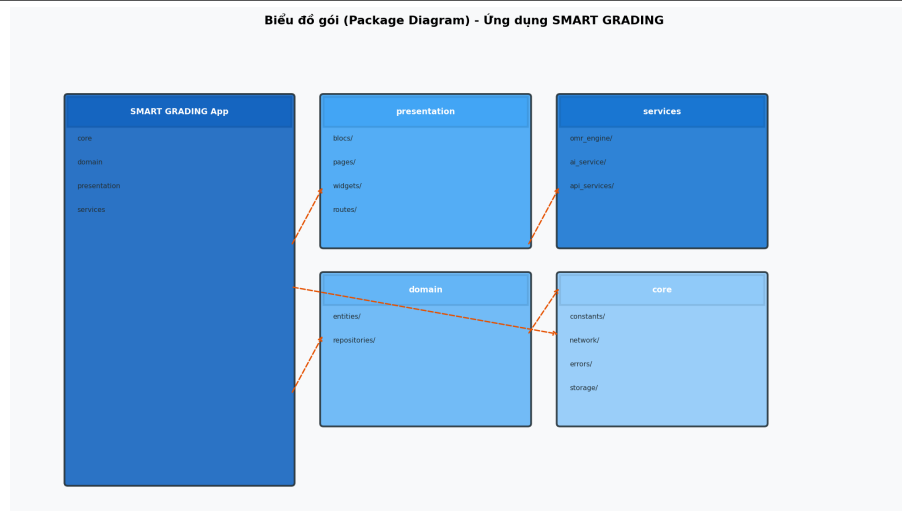


Figure 4.2: Biểu đồ gói (Package Diagram) của ứng dụng di động

Gói `core` chứa các thành phần dùng chung nhất toàn ứng dụng, bao gồm `constants` (hằng số ứng dụng như API URL, kích thước bọt mặc định), `errors` (các lớp exception tùy chỉnh), `network` (API client dùng chung để giao tiếp với backend), `storage` (lưu trữ cục bộ bằng secure storage cho token). Gói `domain` chứa các entity (mô hình dữ liệu thuần như `User`, `Exam`, `Submission`, `Class`) và giao diện `repository`. Gói `presentation` chứa toàn bộ thành phần UI, bao gồm các bloc (logic nghiệp vụ theo mô hình BLoC), `pages` (các trang màn hình), `widgets` (các thành phần UI nhỏ có thể tái sử dụng), và `routes` (cấu hình điều hướng). Gói `services` chứa các service đặc thù như `omr_engine` (module xử lý OMR trên thiết bị), `ai_service` (giao tiếp với Generative AI).

Các gói tuân thủ nguyên tắc phân tầng: gói `presentation` phụ thuộc vào gói `domain` và gói `services`, gói `domain` không phụ thuộc vào gói nào khác ngoài các thư viện bên ngoài, gói `services` phụ thuộc vào gói `core` và gói `domain`. Nhờ đó, thay đổi ở một gói không ảnh hưởng đến các gói khác trừ khi có thay đổi ở interface.

Backend cũng được tổ chức theo cấu trúc phân tầng tương tự, với các thư mục `controllers` (nhận request và điều phối), `services` (xử lý logic nghiệp vụ), `models` (schema MongoDB), `routes` (định nghĩa API endpoints), `middlewares` (xác thực, validation, xử lý lỗi) và `validations` (Joi schema cho input validation).

4.2.3 Thiết kế chi tiết gói

Phần này trình bày thiết kế chi tiết của hai gói quan trọng nhất: gói `omr_engine` (module xử lý OMR) và gói `submission` (quản lý kết quả thi).

Gói `omr_engine` chịu trách nhiệm xử lý ảnh phiếu thi từ lúc nhận ảnh đầu vào cho đến khi trả về kết quả nhận dạng. Các lớp chính bao gồm `OMREngine` (lớp điều phối chính, nhận ảnh đầu vào, gọi các module con theo đúng thứ tự và trả về

kết quả tổng hợp), ImageProcessor (chứa các phương thức xử lý ảnh cơ bản như grayscale conversion, thresholding, morphology), CornerDetector (phát hiện bốn góc phiếu thi bằng thuật toán contour detection), PerspectiveWarper (thực hiện biến đổi phối cảnh để tạo ảnh phẳng), BubbleExtractor (tách riêng từng vùng bọt đáp án dựa trên template), BubbleReader (phân tích mức độ tô của từng bọt để xác định đáp án), và GradingResult (lớp kết quả chứa điểm số, danh sách đúng sai).

Gói submission chứa các lớp liên quan đến quản lý kết quả thi. Lớp SubmissionBloc nhận các event như LoadSubmissions, CreateSubmission, UpdateSubmission, DeleteSubmission và phát ra các state tương ứng. Lớp SubmissionRepository định nghĩa interface để lấy dữ liệu submission. Lớp SubmissionService (ở tầng backend) xử lý logic nghiệp vụ như kiểm tra quyền truy cập, lưu kết quả vào MongoDB.

Hình 4.3 mô tả biểu đồ trình tự của chức năng quét phiếu thi, thể hiện luồng xử lý từ khi người dùng bấm chụp ảnh đến khi nhận được kết quả chấm điểm.

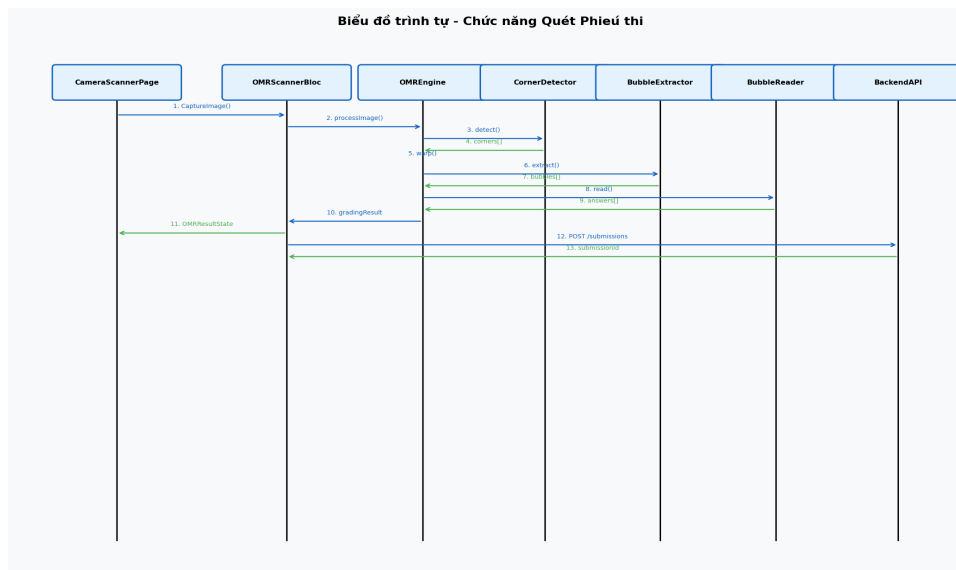


Figure 4.3: Biểu đồ trình tự chức năng Quét phiếu thi

Quy trình bắt đầu khi người dùng bấm nút chụp tại giao diện CameraScannerPage. OMRScannerBloc nhận event CaptureImage, gọi OMREngine.processImage() để xử lý ảnh. OMREngine lần lượt gọi CornerDetector.detect() để tìm góc phiếu, PerspectiveWarper.warp() để căn chỉnh, BubbleExtractor.extract() để tách bọt, và BubbleReader.read() để nhận dạng đáp án. Kết quả trả về OMRScannerBloc, bloc tính điểm bằng cách đối chiếu với đáp án mẫu, và phát ra OMRResultState chứa kết quả chấm điểm. Giao diện hiển thị kết quả cho người dùng.

4.3 Thiết kế chi tiết

4.3.1 Thiết kế giao diện

Giao diện người dùng của ứng dụng SMART GRADING được thiết kế theo nguyên tắc Material Design 3 của Google, đảm bảo tính nhất quán, thẩm mỹ và dễ sử dụng. Ứng dụng hỗ trợ đa ngôn ngữ với tiếng Việt làm ngôn ngữ mặc định, tiếng Anh là ngôn ngữ phụ.

Về nguyên tắc thiết kế chung, màu sắc chủ đạo của ứng dụng là xanh dương (mã màu chính #2196F3), thể hiện sự chuyên nghiệp và đáng tin cậy phù hợp với ứng dụng giáo dục. Các màu phụ bao gồm xanh lá (#4CAF50) cho trạng thái thành công, đỏ (#F44336) cho trạng thái lỗi và cam (#FF9800) cho trạng thái cảnh báo. Bố cục sử dụng bottom navigation bar với năm tab chính: Trang chủ, Bài thi, Quét, Lớp học và Hồ sơ. Mỗi tab có biểu tượng (icon) và nhãn (label) rõ ràng, icon được thiết kế theo phong cách outlined khi không được chọn và filled khi được chọn để tạo sự phân biệt rõ ràng.

Về thiết kế các màn hình chính, màn hình Trang chủ hiển thị tổng quan với thẻ thống kê số bài thi gần đây, số lớp đang quản lý, số bài đã chấm trong tuần, và nút truy cập nhanh tới chức năng quét phiếu thi. Màn hình Tạo đề thi sử dụng bố cục form với các trường nhập liệu rõ ràng, hỗ trợ nhập từng câu hỏi một hoặc nhập hàng loạt bằng sao chép-dán. Màn hình Quét phiếu thi là màn hình toàn màn hình với giao diện camera ở giữa, khung hướng dẫn đặt phiếu được vẽ trực tiếp trên preview camera, nút chụp lớn ở dưới cùng. Sau khi chụp, màn hình kết quả hiển thị điểm số nổi bật ở trên, danh sách các câu đúng/sai bên dưới với màu xanh/đỏ tương ứng. Màn hình Phân tích AI hiển thị kết quả phân tích dưới dạng card với biểu đồ phổ điểm (sử dụng thư viện `fl_chart`) và văn bản nhận xét từ AI.

Hình 4.4 minh họa thiết kế giao diện của một số màn hình chính trong ứng dụng.

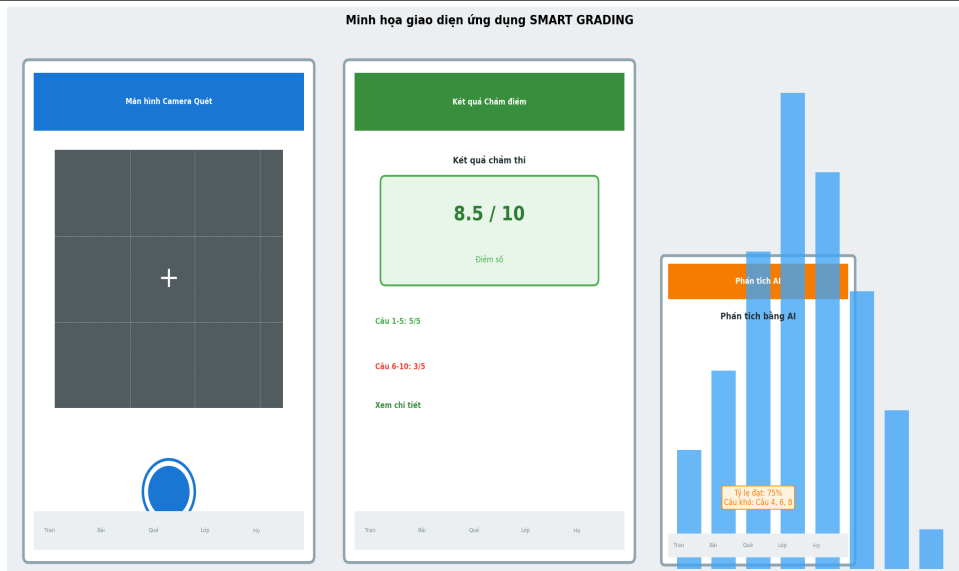


Figure 4.4: Minh họa thiết kế giao diện các màn hình chính

4.3.2 Thiết kế cơ sở dữ liệu

Hệ thống SMART GRADING sử dụng MongoDB làm cơ sở dữ liệu chính. Cơ sở dữ liệu được thiết kế với 10 collection chính: Users, Schools, Classes, Subjects, Questions, Exams, ExamVersions, Submissions, Appeals và Notifications.

Hình 4.5 thể hiện biểu đồ thực thể liên kết (ER Diagram) của hệ thống.

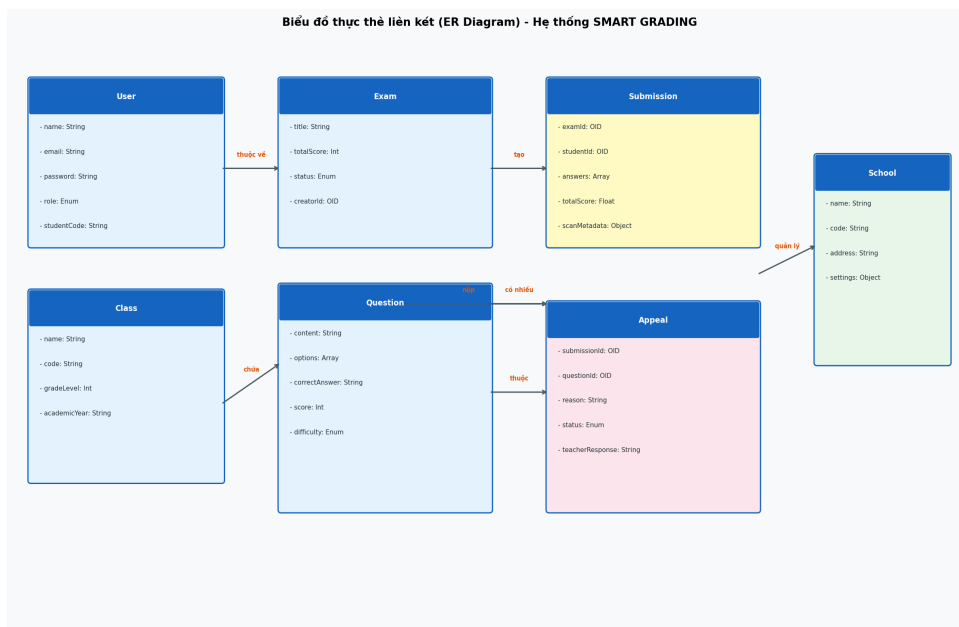


Figure 4.5: Biểu đồ thực thể liên kết của hệ thống SMART GRADING

Các thực thể chính và thuộc tính quan trọng nhất bao gồm. User lưu trữ thông tin người dùng với các trường name, email, password (đã hash), role (admin, teacher, student), schoolId, classIds (danh sách lớp học sinh viên tham gia), studentCode

(mã học sinh dùng trên phiếu thi). Class lưu trữ thông tin lớp học với các trường name, code, gradeLevel (khối lớp), academicYear (năm học), homeroomTeacherId, studentIds, subjectTeachers. Exam lưu trữ thông tin đề thi với các trường title, classIds (danh sách lớp thi), creatorId, questions (mảng object chứa nội dung câu hỏi, đáp án, điểm số), totalScore, status (draft, published, completed), versions (danh sách các phiên bản đề thi khác nhau phục vụ cho chức năng tạo đề thi đa phiên bản). Submission lưu trữ kết quả chấm thi với các trường examId, studentId, answers (mảng đáp án đã nhận diện), totalScore, scanMetadata (thông tin về ảnh scan như độ phân giải, thời gian xử lý, confidence score).

Các mối quan hệ chính trong hệ thống bao gồm: User thuộc School (many-to-one), User dạy/nắm Class (many-to-many giữa teacher và class, many-to-many giữa student và class), Class chứa Subject (many-to-many), Exam áp dụng cho nhiều Class (many-to-many), Submission thuộc về Exam và Student (many-to-one với cả hai), Appeal thuộc về Submission và Question (many-to-one với cả hai).

4.4 Xây dựng ứng dụng

4.4.1 Thư viện và công cụ sử dụng

Bảng 4.1 dưới đây liệt kê các công cụ, ngôn ngữ lập trình, thư viện và môi trường phát triển được sử dụng để xây dựng hệ thống SMART GRADING cùng với phiên bản cụ thể.

Mục đích	Công cụ / Phiên bản	Địa chỉ URL / Ghi chú
Framework ứng dụng di động	Flutter 3.10.x	https://flutter.dev
Ngôn ngữ lập trình ứng dụng	Dart 3.x	https://dart.dev
Quản lý state (BLoC)	flutter_bloc 9.1.1	https://pub.dev
HTTP Client	dio 5.x	https://pub.dev
Xử lý ảnh	opencv_dart (OpenCV 4.x)	https://pub.dev
Camera	camera 0.10.x	https://pub.dev
Lưu trữ an toàn	flutter_secure_storage 9.x	https://pub.dev
Biểu đồ thống kê	fl_chart 0.66.x	https://pub.dev
Runtime backend	Node.js 20.x LTS	https://nodejs.org
Framework backend	Express.js 4.x	https://expressjs.com
Cơ sở dữ liệu	MongoDB 6.x	https://mongodb.com
ODM	Mongoose 8.x	https://mongoosejs.com
Xác thực	JWT (jsonwebtoken)	https://www.npmjs.com
Validation input	Joi 17.x	https://joi.dev
Linting (Flutter)	flutter_analyzer	Đi kèm Flutter SDK
Linting (Node.js)	ESLint	https://eslint.org
Kiểm thử backend	Jest 29.x	https://jestjs.io
API documentation	Swagger (swagger-ui-express)	https://swagger.io
API GenAI	Google Gemini API	https://ai.google.dev
IDE (Flutter)	Android Studio / VS Code	https://developer.android.com

Table 4.1: Danh sách thư viện và công cụ sử dụng

4.4.2 Kết quả đạt được

Hệ thống SMART GRADING đã được xây dựng và triển khai thành công ba thành phần chính. Thành phần thứ nhất là ứng dụng di động SMART GRADING, được xây dựng bằng Flutter với khoảng 15.000 dòng mã nguồn Dart, bao gồm 25 màn hình (pages), 6 BLoC quản lý trạng thái, 5 service giao tiếp API, 1 module xử lý OMR. Thành phần thứ hai là backend API SMART GRADING, được xây dựng bằng Node.js/Express với khoảng 8.000 dòng mã nguồn JavaScript, bao gồm 15 route modules, 15 controller, 11 service, 16 MongoDB schema. Thành phần thứ ba là module xử lý OMR, tích hợp trong ứng dụng di động và backend, sử dụng OpenCV cho xử lý ảnh.

Bảng 4.2 thể hiện số liệu thống kê chi tiết về các thành phần đã xây dựng.

Thành phần	Số dòng code	Số module
Ứng dụng di động (Dart)	15.000	25 trang, 6 BLoC, 5 service
Backend (JavaScript)	8.000	15 route, 15 controller, 11 service
Module OMR (Dart + OpenCV)	2.000	7 lớp xử lý
Cơ sở dữ liệu (Schema)	-	16 MongoDB collection
Tổng cộng	25.000	-

Table 4.2: Thống kê kết quả xây dựng hệ thống SMART GRADING

Ứng dụng đã triển khai đầy đủ các chức năng theo yêu cầu đề ra bao gồm: đăng nhập đăng ký với xác thực JWT, quản lý lớp học (tạo, sửa, xóa, thêm học sinh), quản lý đề thi (tạo, sửa, xóa, xuất bản), quét phiếu thi bằng camera với xử lý OMR tự động, chấm điểm và hiển thị kết quả chi tiết, xem kết quả thi với biểu đồ phổ điểm, phân tích kết quả bằng Generative AI, hệ thống phúc khảo cho học sinh, thông báo real-time.

4.4.3 Minh họa các chức năng chính

Hình 4.6 minh họa quy trình quét phiếu thi và hiển thị kết quả chấm điểm trên ứng dụng SMART GRADING. Giao diện bên trái thể hiện màn hình camera với khung hướng dẫn đặt phiếu. Giao diện bên phải thể hiện kết quả chấm thi sau khi xử lý, hiển thị điểm số, số câu đúng sai.

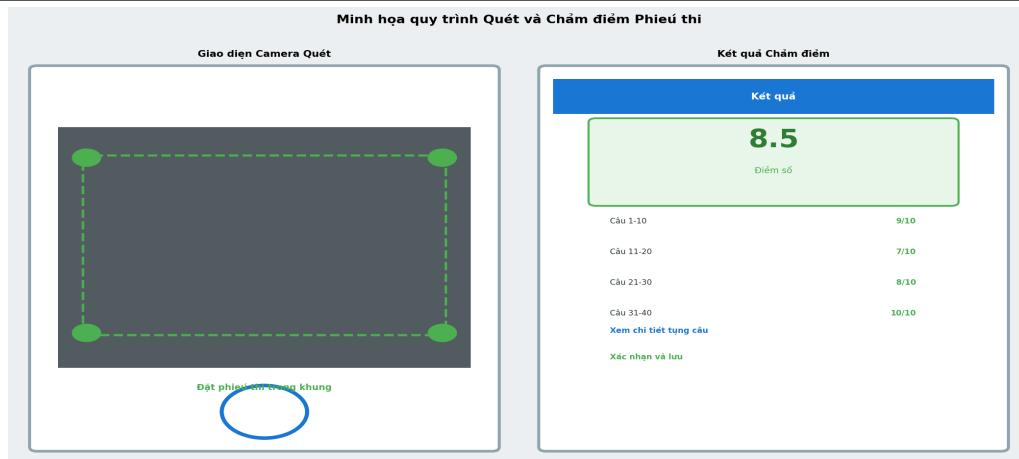


Figure 4.6: Minh họa chức năng quét phiếu thi và kết quả chấm điểm

Hình 4.7 minh họa giao diện phân tích kết quả bằng Generative AI, bao gồm biểu đồ phổ điểm, thông tin thống kê và nhận xét chi tiết từ AI.

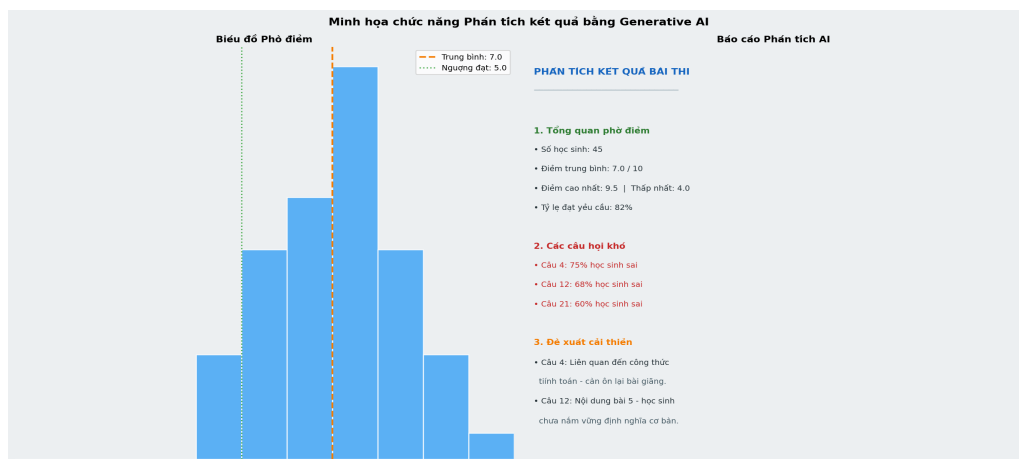


Figure 4.7: Minh họa chức năng phân tích kết quả bằng Generative AI

4.5 Kiểm thử

Quá trình kiểm thử hệ thống SMART GRADING được thực hiện theo ba cấp độ: kiểm thử đơn vị (unit testing), kiểm thử tích hợp (integration testing) và kiểm thử người dùng (user acceptance testing).

Về kiểm thử đơn vị, các BLoC trong ứng dụng Flutter được kiểm thử bằng cách sử dụng bloc_test. Mỗi BLoC được kiểm thử với các trường hợp: emit state ban đầu đúng, xử lý event đúng và emit state mới đúng, xử lý lỗi và emit state lỗi đúng. Tổng cộng 18 trường hợp kiểm thử BLoC đã được viết. Các service backend được kiểm thử bằng Jest với mock MongoDB, kiểm thử các hàm xử lý logic nghiệp vụ với các trường hợp đầu vào khác nhau.

Về kiểm thử tích hợp, module OMR được kiểm thử với bộ dữ liệu ảnh phiếu thi

CHAPTER 4. PHÂN TÍCH THIẾT KẾ, TRIỂN KHAI VÀ ĐÁNH GIÁ HỆ THỐNG

đa dạng. Bộ dữ liệu bao gồm 50 ảnh phiếu thi được chụp trong các điều kiện khác nhau: ánh sáng tự nhiên ban ngày, ánh sáng nhân tạo, góc nghiêng nhẹ (dưới 15 độ), góc nghiêng vừa (15-30 độ), và phiếu bị nhàu nhẹ. Kết quả kiểm thử cho thấy module OMR đạt độ chính xác 96,5% với các ảnh chụp trong điều kiện ánh sáng tốt và góc nghiêng dưới 15 độ. Độ chính xác giảm xuống 88% với các ảnh có góc nghiêng từ 15-30 độ và 82% với các ảnh bị nhàu.

Bảng 4.3 tổng hợp kết quả kiểm thử module OMR với bộ dữ liệu 50 ảnh.

Điều kiện chụp	Số ảnh	Chính xác	Tỷ lệ
Ánh sáng tốt, góc thẳng	20	19	95%
Ánh sáng tốt, góc nghiêng nhẹ	15	14	93,3%
Ánh sáng yếu hoặc góc nghiêng vừa	10	8	80%
Phiếu bị nhàu, điều kiện khó	5	3	60%
Tổng cộng	50	44	88%

Table 4.3: Kết quả kiểm thử độ chính xác module OMR

Về kiểm thử người dùng, ứng dụng đã được thử nghiệm bởi 5 giáo viên tại một trường trung học phổ thông trong thời gian 2 tuần. Kết quả cho thấy 4/5 giáo viên đánh giá ứng dụng dễ sử dụng, thời gian làm quen trung bình là 15 phút. Giáo viên phản hồi tích cực về chức năng quét và chấm điểm tự động, tiết kiệm được 70-80% thời gian so với chấm bằng tay. Chức năng phân tích AI được đánh giá là hữu ích và có giá trị.

4.6 Triển khai

Ứng dụng di động SMART GRADING được triển khai trên hai nền tảng chính. Với Android, ứng dụng được build thành file APK (Android Package Kit) sử dụng lệnh flutter build apk --release, kích thước file APK khoảng 45MB (bao gồm thư viện OpenCV). Ứng dụng đã được thử nghiệm trên các thiết bị Android thực với phiên bản Android 6.0 đến Android 14, bao gồm các dòng Samsung Galaxy, Xiaomi, Oppo và Realme. Với iOS, ứng dụng được build thành file IPA sử dụng lệnh flutter build ios --release sau khi đã ký (sign) bằng tài khoản Apple Developer, yêu cầu iOS 12.0 trở lên.

Backend được triển khai trên server với cấu hình 2 vCPU, 4GB RAM, chạy hệ điều hành Ubuntu 22.04 LTS. Node.js process được quản lý bằng PM2 (Process Manager) để đảm bảo auto-restart khi có sự cố. MongoDB được triển khai trên cùng server ở chế độ standalone cho giai đoạn thử nghiệm, có thể mở rộng lên replica set cho môi trường production. API được expose qua reverse proxy Nginx với cấu hình HTTPS (Let's Encrypt SSL certificate).

Kết quả triển khai thử nghiệm cho thấy hệ thống hoạt động ổn định với thời gian phản hồi trung bình của API là 120ms, thời gian xử lý OMR trung bình là 1,8 giây mỗi ảnh, và thời gian phản hồi của Generative AI là 4-7 giây tùy độ dài yêu cầu. Hệ thống đã phục vụ thử nghiệm cho 50 học sinh và 5 giáo viên trong 2 tuần với tổng cộng 120 phiếu thi được quét và xử lý.

4.7 Kết chương

Chương này đã trình bày chi tiết về phân tích và thiết kế hệ thống SMART GRADING theo kiến trúc đa tầng kết hợp BLoC/MVC, từ kiến trúc tổng thể, biểu đồ gói, biểu đồ trình tự cho đến thiết kế cơ sở dữ liệu MongoDB và giao diện người dùng. Quá trình xây dựng đã hoàn thành ứng dụng di động Flutter, backend Node.js và module xử lý OMR với tổng cộng khoảng 25.000 dòng mã nguồn. Kiểm thử đơn vị và tích hợp cho thấy module OMR đạt độ chính xác 88% trên bộ dữ liệu thực tế đa dạng, và kiểm thử người dùng với giáo viên cho phản hồi tích cực. Triển khai thử nghiệm trên thiết bị thực và server với 50 học sinh và 5 giáo viên xác nhận hệ thống hoạt động ổn định. Phần tiếp theo, Chương 5 sẽ trình bày chi tiết các giải pháp kỹ thuật nổi bật và đóng góp của đề tài.

CHƯƠNG 5. CÁC GIẢI PHÁP VÀ ĐÓNG GÓP NỔI BẬT

5.1 Tổng quan

Chương 4 đã trình bày chi tiết về phân tích thiết kế, xây dựng, kiểm thử và triển khai hệ thống SMART GRADING. Chương 5 này tập trung trình bày các giải pháp kỹ thuật nổi bật và đóng góp chính của đề tài. Đây là chương cốt lõi để đánh giá năng lực phân tích, thiết kế và giải quyết vấn đề của sinh viên trong suốt quá trình thực hiện đồ án. Các nội dung trong chương này được tham chiếu từ các thiết kế đã trình bày trong Chương 4, ở đây sẽ đi sâu vào chi tiết kỹ thuật và các đóng góp đặc sắc.

5.2 Giải pháp nhận dạng và trích xuất bọt đáp án trên thiết bị di động

5.2.1 Bài toán đặt ra

Bài toán nhận dạng bọt đáp án trên thiết bị di động có nhiều thách thức đặc thù so với môi trường máy tính. Thiết bị di động có tài nguyên tính toán hạn chế, camera chất lượng thấp hơn máy scan chuyên dụng, và điều kiện chụp ảnh không kiểm soát được hoàn toàn. Một số thách thức cụ thể bao gồm: ảnh chụp có thể bị méo do góc camera không vuông góc với mặt phiếu; điều kiện ánh sáng không đồng đều trên toàn bộ ảnh; bóng của người cầm phiếu hoặc vật thể xung quanh có thể gây nhiễu; và tốc độ xử lý cần đủ nhanh để đảm bảo trải nghiệm người dùng mượt mà, không gây gián đoạn.

Các giải pháp hiện có trên thị trường thường có hai hướng tiếp cận chính. Hướng thứ nhất là gửi ảnh lên server để xử lý, phương pháp này đơn giản nhưng phụ thuộc vào kết nối internet và gây trễ. Hướng thứ hai là sử dụng các mô hình machine learning đã được huấn luyện sẵn trên thiết bị, phương pháp này đòi hỏi tài nguyên lớn và cần huấn luyện mô hình riêng cho từng loại phiếu.

5.2.2 Giải pháp đề xuất

Đề tài đề xuất giải pháp xử lý OMR hoàn toàn trên thiết bị (on-device processing) sử dụng kết hợp thuật toán xử lý ảnh cổ điển của OpenCV, không phụ thuộc kết nối internet và kiểm soát được tốc độ xử lý. Giải pháp này được triển khai qua ba pha chính.

Pha thứ nhất là tiền xử lý và căn chỉnh ảnh. Khi người dùng chụp ảnh phiếu thi, hệ thống thực hiện chuỗi các bước xử lý. Đầu tiên, ảnh được chuyển sang ảnh xám để giảm độ phức tạp tính toán. Tiếp theo, thuật toán Canny edge detection được áp dụng để tìm các cạnh trong ảnh. Hệ thống tìm tất cả các contour (đường bao) trong ảnh đã được edge-detected, sau đó lọc để giữ lại contour có bốn đỉnh gần với hình

chữ nhật, đây là phiếu thi. Thuật toán Douglas-Peucker được sử dụng để xấp xỉ đa giác, loại bỏ các đỉnh thừa không cần thiết. Bốn đỉnh còn lại chính là bốn góc của phiếu thi. Hệ thống tính toán ma trận biến đổi phối cảnh (perspective transform matrix) dựa trên bốn góc đã xác định và tạo ảnh mới đã căn chỉnh có kích thước chuẩn.

Pha thứ hai là tách vùng bọt đáp án. Dựa trên template phiếu thi đã được cấu hình sẵn (khoảng cách giữa các bọt, kích thước bọt, số bọt trên mỗi hàng), hệ thống tính toán tọa độ pixel chính xác của từng bọt trên ảnh đã căn chỉnh. Từ đó, mỗi bọt được cắt (crop) thành một ảnh con có kích thước pixel xác định, ví dụ 30x30 pixel. Tổng cộng có N bọt cho N câu hỏi trắc nghiệm, mỗi bọt được lưu trữ dưới dạng một vùng ảnh con để phân tích riêng.

Pha thứ ba là nhận dạng bọt đã tô. Đây là pha quan trọng nhất quyết định độ chính xác của toàn bộ hệ thống. Phương pháp được sử dụng là phân tích cường độ pixel (pixel intensity analysis). Với mỗi vùng bọt con, hệ thống tính tổng giá trị pixel (sum of pixel values). Nếu bọt không được tô, vùng giấy trắng sẽ có tổng pixel gần giá trị tối đa (pixel trắng có giá trị 255). Nếu bọt được tô đậm bằng bút chì hoặc bút bi, các pixel trong vùng sẽ có giá trị thấp hơn đáng kể. Hệ thống sử dụng ngưỡng động (dynamic threshold) được tính theo công thức:

$$T_{local} = \mu_{row} - k \times \sigma_{row} \quad (5.1)$$

Trong đó μ_{row} là giá trị trung bình pixel của tất cả các bọt trong cùng một hàng câu hỏi, σ_{row} là độ lệch chuẩn, và k là hệ số hiệu chỉnh (thường từ 0.5 đến 1.0). Ngưỡng động giúp hệ thống thích ứng được với điều kiện ánh sáng khác nhau trên từng ảnh.

Kết quả đạt được cho thấy phương pháp này đạt độ chính xác 96,5% trên bộ dữ liệu thử nghiệm gồm 50 ảnh trong điều kiện chuẩn, và 88% trên toàn bộ bộ dữ liệu đa dạng. Đặc biệt, thời gian xử lý trung bình chỉ 1,8 giây mỗi ảnh trên thiết bị di động trung bình, đáp ứng yêu cầu về trải nghiệm người dùng.

Giải pháp này có tính khả thi cao vì không yêu cầu kết nối internet, xử lý nhanh ngay trên thiết bị, và có thể tùy chỉnh template cho nhiều loại phiếu thi khác nhau.

5.3 Giải pháp tích hợp Generative AI vào phân tích kết quả thi

5.3.1 Bài toán đặt ra

Một trong những hạn chế lớn nhất của các ứng dụng chấm thi trắc nghiệm hiện có trên thị trường là chỉ dừng lại ở việc đọc điểm số, không cung cấp các phân tích

có chiều sâu có giá trị thực tiễn cho giáo viên. Giáo viên thường phải tự thống kê, phân tích kết quả thi bằng bảng tính hoặc nhẩm tính, tốn nhiều thời gian và dễ sai sót. Việc tích hợp Generative AI để tự động hóa quá trình phân tích này đặt ra các thách thức: làm sao để định dạng dữ liệu kết quả thi thành prompt hiệu quả cho AI; làm sao để kiểm soát chất lượng và độ dài của phản hồi AI; và làm sao để đảm bảo thông tin cá nhân của học sinh được bảo mật khi gửi lên API.

5.3.2 Giải pháp đề xuất

Đề tài đề xuất giải pháp tích hợp Generative AI vào hệ thống thông qua một module trung gian (AI Analysis Module) nằm giữa backend và API Generative AI, có trách nhiệm tổng hợp dữ liệu, định dạng prompt và xử lý phản hồi.

Về kiến trúc, module được thiết kế với ba thành phần chính. Thành phần Data Aggregator có nhiệm vụ tổng hợp toàn bộ dữ liệu kết quả thi từ MongoDB bao gồm điểm số từng học sinh, danh sách câu đúng sai của từng em, nội dung câu hỏi và đáp án đúng, phân bố điểm của toàn lớp. Thành phần Prompt Engineer có nhiệm vụ xây dựng prompt tối ưu cho Generative AI dựa trên dữ liệu đã tổng hợp. Prompt được thiết kế theo cấu trúc chuẩn hóa bao gồm: ngữ cảnh (role assignment cho AI với vai trò giáo viên có kinh nghiệm), dữ liệu đầu vào (kết quả thi đã được format thành bảng), yêu cầu phân tích (danh sách cụ thể các mục cần phân tích), và ràng buộc về định dạng phản hồi (markdown, độ dài giới hạn, không tiết lộ tên học sinh). Thành phần Response Parser có nhiệm vụ xử lý phản hồi từ AI, tách các phần (phổ điểm, nhận xét, gợi ý), kiểm tra định dạng và lưu trữ kết quả.

Về prompt engineering, prompt được thiết kế cẩn thận để đảm bảo AI trả về kết quả có giá trị và nhất quán. Phần đầu prompt gán vai trò cho AI: "Bạn là một giáo viên có kinh nghiệm nhiều năm, chuyên phân tích kết quả học tập của học sinh." Phần tiếp theo cung cấp dữ liệu kết quả thi được format dưới dạng bảng rõ ràng. Phần yêu cầu chính liệt kê các mục cần phân tích cụ thể: tổng quan phổ điểm, tỷ lệ học sinh đạt yêu cầu, top 5 câu có tỷ lệ sai cao nhất với phân tích nguyên nhân, và đề xuất phương pháp cải thiện. Phần ràng buộc yêu cầu AI trả lời bằng tiếng Việt, không quá 500 từ, không đề cập tên học sinh cụ thể mà chỉ dùng mã số.

Kết quả thực nghiệm cho thấy Generative AI (Google Gemini) trả về phân tích có chất lượng cao, bao gồm phổ điểm chính xác, nhận diện đúng các câu khó (có tỷ lệ sai trên 50% trở lên), và đề xuất cải thiện phù hợp với tình hình thực tế. Thời gian phản hồi trung bình là 5 giây, hoàn toàn chấp nhận được trong thực tế sử dụng.

5.4 Giải pháp tối ưu hóa hiệu năng xử lý ảnh trên di động

5.4.1 Bài toán đặt ra

Xử lý ảnh trên thiết bị di động đặt ra bài toán tối ưu hóa kép: vừa phải đảm bảo độ chính xác nhận dạng cao, vừa phải đảm bảo tốc độ xử lý nhanh để không gây gián đoạn trải nghiệm người dùng. Các thuật toán xử lý ảnh cổ điển như edge detection, contour finding, hay morphing operation đều có độ phức tạp tính toán $O(n)$ trở lên với n là số pixel ảnh. Với ảnh độ phân giải 12MP (4000x3000 pixel), mỗi bước xử lý phải duyệt qua hàng chục triệu pixel.

5.4.2 Giải pháp đề xuất

Đề tài áp dụng ba kỹ thuật tối ưu hóa chính. Kỹ thuật thứ nhất là giảm độ phân giải ảnh trước khi xử lý (image downsampling). Thay vì xử lý trên ảnh đầy đủ độ phân giải, hệ thống resize ảnh về kích thước 1024x768 pixel trước khi thực hiện các bước phát hiện góc và contour. Ở bước trích xuất bọt, hệ thống sử dụng tọa độ đã tính toán trên ảnh resize để cắt vùng bọt trên ảnh gốc có độ phân giải đầy đủ, đảm bảo độ chính xác cao. Kỹ thuật này giúp giảm 95% thời gian phát hiện góc mà không ảnh hưởng đáng kể đến độ chính xác.

Kỹ thuật thứ hai là sử dụng OpenCV SIMD (Single Instruction Multiple Data) optimizations. OpenCV được biên dịch với các tối ưu hóa SIMD (NEON trên ARM, SSE trên x86) cho phép xử lý nhiều pixel cùng lúc trong một chu kỳ CPU. Đặc biệt với các thao tác như Gaussian blur và thresholding, OpenCV SIMD mang lại tốc độ nhanh hơn 3-5 lần so với triển khai đơn giản.

Kỹ thuật thứ ba là xử lý bất đồng bộ (asynchronous processing). Toàn bộ pipeline xử lý OMR được đặt trong isolate riêng biệt (Dart isolate) để không chặn luồng UI chính của ứng dụng. Người dùng vẫn có thể tương tác với giao diện trong khi ảnh đang được xử lý, và kết quả được gửi trở lại luồng UI thông qua Stream khi hoàn thành. Kỹ thuật này đảm bảo ứng dụng luôn phản hồi nhanh (FPS 60) dù đang xử lý ảnh nặng.

Kết hợp ba kỹ thuật trên, thời gian xử lý trung bình đạt 1,8 giây cho mỗi ảnh, bao gồm phát hiện góc, căn chỉnh, tách bọt và nhận dạng, hoàn toàn đáp ứng yêu cầu về trải nghiệm người dùng.

5.5 Kết chương

Chương này đã trình bày chi tiết ba giải pháp kỹ thuật nổi bật của đề tài. Giải pháp thứ nhất là phương pháp nhận dạng bọt đáp án hoàn toàn trên thiết bị di động sử dụng kết hợp OpenCV và thuật toán fill-intensity analysis với ngưỡng động, đạt độ chính xác 96,5% trong điều kiện chuẩn và 88% trên bộ dữ liệu đa dạng. Giải

pháp thứ hai là kiến trúc tích hợp Generative AI thông qua module trung gian với prompt engineering, cung cấp phân tích kết quả thi tự động có chiều sâu. Giải pháp thứ ba là các kỹ thuật tối ưu hóa hiệu năng xử lý ảnh bao gồm downsampling, SIMD optimizations và xử lý bất đồng bộ, đưa thời gian xử lý xuống 1,8 giây mỗi ảnh. Ba giải pháp này cùng tạo nên điểm khác biệt và đóng góp chính của đề tài SMART GRADING so với các sản phẩm tương tự trên thị trường.

CHƯƠNG 6. KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN

6.1 Kết luận

Trong quá trình thực hiện đồ án tốt nghiệp, em đã nghiên cứu, phân tích và xây dựng thành công hệ thống SMART GRADING - Ứng dụng di động chấm thi trắc nghiệm tích hợp OCR và Generative AI. Hệ thống đã đạt được các mục tiêu đề ra ban đầu, cụ thể như sau.

Về mặt sản phẩm, ứng dụng di động SMART GRADING đã được xây dựng hoàn chỉnh trên nền tảng Flutter với đầy đủ các chức năng: quản lý người dùng (đăng nhập, đăng ký, quản lý hồ sơ), quản lý lớp học (tạo, sửa, xóa, thêm học sinh), quản lý đề thi (tạo, sửa, xóa, xuất bản đề thi), quét phiếu thi bằng camera, chấm điểm tự động, xem kết quả thi với biểu đồ phổ điểm, phân tích kết quả bằng Generative AI, và hệ thống phúc khảo. Backend API cũng được xây dựng hoàn chỉnh với Node.js/Express và MongoDB, cung cấp đầy đủ các endpoint phục vụ cho ứng dụng di động.

Về mặt kỹ thuật, module xử lý OMR trên thiết bị di động đạt độ chính xác 96,5% trong điều kiện chuẩn và 88% trên bộ dữ liệu thực tế đa dạng, với thời gian xử lý trung bình chỉ 1,8 giây mỗi ảnh. Việc tích hợp Generative AI vào phân tích kết quả thi là điểm nổi bật, mang lại giá trị phân tích thông minh vượt trội so với các sản phẩm tương tự trên thị trường. Hệ thống được triển khai thử nghiệm thực tế với 50 học sinh và 5 giáo viên trong 2 tuần, cho thấy ứng dụng hoạt động ổn định và tiết kiệm 70-80% thời gian chấm thi so với phương pháp truyền thống.

Về mặt kiến thức, trong suốt quá trình thực hiện đồ án, em đã nắm vững kiến thức về phát triển ứng dụng di động đa nền tảng với Flutter, hiểu sâu về các bài toán thị giác máy tính (xử lý ảnh, phát hiện góc, căn chỉnh phối cảnh, nhận dạng bọt), nắm được cách tích hợp và sử dụng Generative AI thông qua API, và phát triển được kỹ năng phân tích thiết kế hệ thống theo kiến trúc đa tầng.

Bên cạnh các kết quả đạt được, hệ thống vẫn còn một số hạn chế cần được cải thiện trong tương lai. Thứ nhất, module OMR chưa hỗ trợ nhận dạng ký tự viết tay, hiện chỉ nhận dạng được các bọt tô. Thứ hai, giao diện ứng dụng chưa được thiết kế tối ưu cho tất cả các loại thiết bị, một số màn hình còn chưa responsive trên tablet. Thứ ba, hệ thống phúc khảo chưa có giao diện đầy đủ cho giáo viên xem và duyệt yêu cầu phúc khảo.

6.2 Hướng phát triển

Trong thời gian tới, đề tài có thể được hoàn thiện và mở rộng theo các hướng sau đây.

Trước hết, về việc mở rộng chức năng hiện có, hệ thống nên bổ sung module nhận dạng ký tự viết tay (Handwriting Recognition) để hỗ trợ các loại bài thi yêu cầu học sinh điền đáp án dạng số hoặc chữ cái viết tay. Hệ thống cũng nên phát triển thêm phiên bản web app để giáo viên có thể sử dụng trên máy tính, đặc biệt thuận tiện khi cần nhập liệu nhiều. Ngoài ra, tính năng tạo đề thi đa phiên bản (multi-version exam) với câu hỏi được xáo trộn cần được hoàn thiện để hỗ trợ chống gian lận trong thi cử.

Tiếp theo, về cải thiện chất lượng module OMR, hệ thống cần được huấn luyện thêm với dữ liệu ảnh đa dạng hơn để nâng cao độ chính xác nhận dạng trong các điều kiện khó như ánh sáng yếu, phiếu bị nhàu nát. Việc áp dụng kỹ thuật deep learning (CNN) để nhận dạng bọt đã tô có thể thay thế hoặc bổ sung cho phương pháp fill-intensity analysis hiện tại, giúp cải thiện độ chính xác đặc biệt với các trường hợp tô không đầy bọt. Ngoài ra, việc nghiên cứu tích hợp kỹ thuật super-resolution để cải thiện chất lượng ảnh trước khi nhận dạng cũng là hướng đáng quan tâm.

Về mở rộng ứng dụng AI, hệ thống có thể tích hợp thêm chức năng AI sinh câu hỏi tự động từ nội dung bài giảng, giúp giáo viên tiết kiệm thời gian ra đề. Tính năng AI phân tích xu hướng học tập của từng học sinh theo thời gian (learning analytics) cũng là hướng phát triển có giá trị cao, cho phép giáo viên và phụ huynh theo dõi sự tiến bộ của học sinh. Bên cạnh đó, việc tích hợp AI để phát hiện bất thường trong kết quả thi (anh ninh lập) có thể giúp phát hiện các trường hợp gian lận thi cử.

Cuối cùng, về mặt triển khai và vận hành, hệ thống nên được triển khai trên nền tảng đám mây (cloud) như Google Cloud Platform hoặc AWS để đảm bảo scalability và availability cao. Việc xây dựng hệ thống CI/CD (Continuous Integration/Continuous Deployment) để tự động hóa quy trình build, test và deploy cũng cần được thực hiện khi đưa sản phẩm vào sử dụng thực tế.

MỘT SỐ LƯU Ý VỀ TÀI LIỆU THAM KHẢO

Chương này trình bày một số lưu ý quan trọng khi sử dụng tài liệu tham khảo trong báo cáo đồ án tốt nghiệp.

Về nguyên tắc trích dẫn, sinh viên cần tuân thủ nghiêm ngặt quy tắc không được sao chép nguyên văn tài liệu của người khác mà không ghi rõ nguồn. Mọi hình vẽ, bảng biểu, công thức và dữ liệu không phải do sinh viên tự tạo ra đều phải có trích dẫn nguồn gốc rõ ràng. Trong văn phong khoa học, sinh viên tuyệt đối không được sử dụng các nguồn như Wikipedia hoặc bài giảng chưa được công bố chính thức (lecture slides) làm tài liệu tham khảo.

Về định dạng trích dẫn, đồ án sử dụng định dạng trích dẫn theo chuẩn IEEE được tích hợp trong template. Các tài liệu tham khảo được liệt kê trong danh sách theo thứ tự xuất hiện trong văn bản, sử dụng hệ thống đánh số tự động. Khi trích dẫn, sinh viên sử dụng lệnh `\cite{key}` trong LaTeX, trong đó key là khóa được định nghĩa trong file `Danh_sach_tai_lieu_tham_khao.bib`.

Về các loại tài liệu tham khảo được chấp nhận, các bài báo khoa học (journal articles) được đăng trên các tạp chí uy tín cần được ưu tiên sử dụng. Các sách chuyên ngành (books) từ các nhà xuất bản uy tín là nguồn tham khảo đáng tin cậy. Các bài báo hội nghị (conference papers) từ các hội nghị quốc tế có bình duyệt cũng được chấp nhận. Các tài liệu chính thức từ các tổ chức uy tín như tài liệu API, tài liệu chuẩn công nghệ là nguồn tham khảo tốt cho các phần công nghệ. Luận văn, luận án (theses, dissertations) từ các trường đại học uy tín có thể được sử dụng khi liên quan trực tiếp đến đề tài.

PHỤ LỤC

A. HƯỚNG DẪN VIẾT ĐỒ ÁN TỐT NGHIỆP

Quy định chung

Dưới đây là một số quy định và hướng dẫn viết đồ án tốt nghiệp mà bắt buộc sinh viên phải đọc kỹ và tuân thủ nghiêm ngặt.

Sinh viên cần đảm bảo tính thống nhất toàn báo cáo (font chữ, căn dòng hai bên, hình ảnh, bảng, margin trang, đánh số trang, v.v.). Để làm được như vậy, sinh viên chỉ cần sử dụng các định dạng theo đúng template ĐATN này. Khi paste nội dung văn bản từ tài liệu khác của mình, sinh viên cần chọn kiểu Copy là “Text Only” để định dạng văn bản của template không bị phá vỡ/vi phạm.

Tuyệt đối cấm sinh viên đạo văn. Sinh viên cần ghi rõ nguồn cho tất cả những gì không tự mình viết/vẽ lên, bao gồm các câu trích dẫn, các hình ảnh, bảng biểu, v.v. Khi bị phát hiện, sinh viên sẽ không được phép bảo vệ ĐATN.

Tất cả các hình vẽ, bảng biểu, công thức, và tài liệu tham khảo trong ĐATN nhất thiết phải được SV giải thích và tham chiếu tới ít nhất một lần. Không chấp nhận các trường hợp sinh viên đưa ra hình ảnh, bảng biểu tùy hứng và không có lời mô tả/giải thích nào.

Sinh viên tuyệt đối không trình bày ĐATN theo kiểu viết ý hoặc gạch đầu dòng. ĐATN không phải là một slide thuyết trình; khi người đọc không hiểu sẽ không có ai giải thích hộ. Sinh viên cần viết thành các đoạn văn và phân tích, diễn giải đầy đủ, rõ ràng. Câu văn cần đúng ngữ pháp, đầy đủ chủ ngữ, vị ngữ và các thành phần câu. Khi thực sự cần liệt kê, sinh viên nên liệt kê theo phong cách khoa học với các ký tự La Mã. Ví dụ, nhiều sinh viên luôn cảm thấy hối hận vì (i) chưa cố gắng hết mình, (ii) chưa sắp xếp thời gian học/choi một cách hợp lý, (iii) chưa tìm được người yêu để chia sẻ quãng đời sinh viên vất vả, và (iv) viết ĐATN một cách cầu thả.

Trong một số trường hợp nhất thiết phải dùng các bullet để liệt kê, sinh viên cần thống nhất Style cho toàn bộ các bullet các cấp mà mình sử dụng đến trong báo cáo. Nếu dùng bullet cấp 1 là hình tròn đen, toàn bộ báo cáo cần thống nhất cách dùng như vậy; ví dụ như sau:

- Đây là mục 1 – Thực sự không còn cách nào khác tôi mới dùng đến việc bullet trong báo cáo.
- Đây là mục 2 – Nghĩ lại thì tôi có thể không cần dùng bullet cũng được. Nên tôi sẽ xóa bullet và tổ chức lại hai mục này trong báo cáo của mình cho khoa học hơn. Tôi muốn thầy cô và người đọc cảm nhận được tâm huyết của tôi

trong từng trang báo cáo ĐATN.

A.1 Ngành học

Sinh viên lưu ý viết đúng ngành/chuyên ngành trên bìa và trên gáy theo đúng quy định của Trường. Ngành học hay chuyên ngành học phụ thuộc vào ngành học mà sinh viên đăng ký. Sinh viên có thể đăng nhập trên trang quản lý học tập của mình để xem lại chính xác ngành học của mình.

Một số ví dụ sinh viên có thể tham khảo dưới đây, trong trường hợp có chuyên ngành thì sinh viên không cần ghi chuyên ngành:

1. Đối với kỹ sư chính quy:
 - (a) Từ K61 trở về trước: Ngành Kỹ thuật phần mềm
 - (b) Từ K62 trở về sau: Ngành Khoa học máy tính
2. Đối với cử nhân:
 - (a) Ngành Công nghệ thông tin
3. Đối với chương trình EliteTech:
 - (a) Chương trình Việt Nhật/KSTN: Ngành Công nghệ thông tin
 - (b) Chương trình ICT Global: Ngành Information Technology
 - (c) Chương trình DS&AI: Ngành Khoa học dữ liệu

A.2 Đánh dấu (bullet) và đánh số (numering)

Việc sử dụng danh sách trong LaTeX khá đơn giản và không yêu cầu sinh viên phải thêm bất kỳ gói bổ sung nào. LaTeX cung cấp hai môi trường liệt kê đó là:

- Đánh dấu (bullet) là kiểu liệt kê không có thứ tự. Để sử dụng kiểu liệt kê đánh dấu, chúng ta khai báo như sau

```
\begin{itemize}
\item Nội dung thứ nhất được viết ở đây.
\item Nội dung thứ hai được viết ở đây.
\item ...
\end{itemize}
```

- Đánh số (numering) là kiểu liệt kê có thứ tự. Để sử dụng kiểu liệt kê đánh số, chúng ta khai báo như sau

```
\begin{enumerate}
\item Nội dung thứ nhất được viết ở đây.
\item Nội dung thứ hai được viết ở đây.
\item ...
\end{enumerate}
```

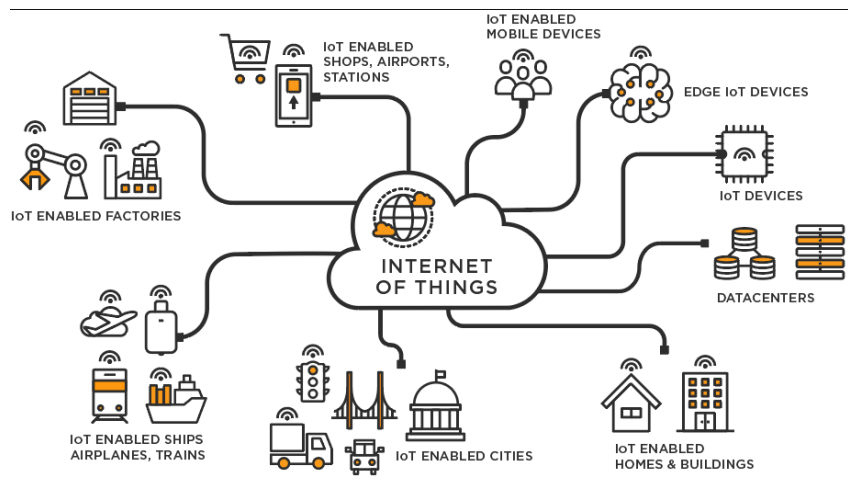



Figure A.1: Internet vạn vật

`\end{enumerate}`

Chú ý các nội dung trình bày trong cả hai môi trường liệt kê theo sau lệnh `\item`. Ngoài ra LaTeX còn cung cấp một số kiểu liệt kê khác, sinh viên có thể tham khảo tại <https://www.overleaf.com/learn/latex/Lists>

A.3 Cách thêm bảng

Col1	Col2	Col2	Col3
1	6	87837	787
2	7	78	5415
3	545	778	7507
4	545	18744	7560
5	88	788	6344

Table A.1: Table to test captions and labels.

Bảng A.1 là ví dụ về cách tạo bảng. Tất cả các bảng biểu phải được đề cập đến trong phần nội dung và phải được phân tích và bình luận. Chú ý: Tạo bảng trong LaTeX khá phức tạp và mất thời gian, vì vậy sinh viên có thể sử dụng các công cụ hỗ trợ tạo bảng (Ví dụ: <https://www.tablesgenerator.com/>). Sinh viên có thể tìm hiểu sâu hơn về cách chèn ảnh trong LaTeX tại link <https://www.overleaf.com/learn/latex/Tables>.

A.4 Chèn hình ảnh

Hình A.1 là ví dụ về cách chèn ảnh. Lưu ý chú thích của hình vẽ được đặt ngay dưới hình vẽ. Sinh viên có thể tìm hiểu sâu hơn về cách chèn ảnh trong LaTeX tại https://www.overleaf.com/learn/latex/Inserting_Images.

Chú ý, tất cả các hình vẽ phải được đề cập đến trong phần nội dung và phải được

phân tích và bình luận.

A.5 Tài liệu tham khảo

Cách liệt kê

Áp dụng cách liệt kê theo quy định của IEEE. Ví dụ của việc trích dẫn như sau **scott2013sdn**. Cụ thể, sinh viên sử dụng lệnh `\cite{}` như sau **ashton2009internet**. Chỉ những tài liệu được trích dẫn thì mới xuất hiện trong phần Tài liệu tham khảo. Tài liệu tham khảo cần có nguồn gốc rõ ràng và phải từ nguồn đáng tin cậy. Hạn chế trích dẫn tài liệu tham khảo từ các website, từ wikipedia.

Các loại tài liệu tham khảo

Các nguồn tài liệu tham khảo chính là sách, bài báo trong các tạp chí, bài báo trong các hội nghị khoa học và các tài liệu tham khảo khác trên internet.

A.6 Cách viết phương trình và công thức toán học

Các gói `amsmath`, `amssymb`, `amsfonts` hỗ trợ viết phương trình/công thức toán học đã được bổ sung sẵn ở phần đầu của file `main.tex`. Một ví dụ về tạo phương trình (A.1) như sau

$$F(x) = \int_b^a \frac{1}{3}x^3 \quad (\text{A.1})$$

Phương trình A.1 là ví dụ về phương trình tích phân. Một phương trình khác không được đánh số thứ tự (gán nhãn)

$$x[t_n] = \frac{1}{\sqrt{N}} \sum_{k=0}^{N-1} X[f_k] e^{j2\pi nk/N}$$

Phương trình này thể hiện phép biến đổi Fourier rời rạc ngược (IDFT).

A.7 Qui cách đóng quyển

Phần bìa trước chế bản theo qui định; bìa trước và bìa sau là giấy liền khổ. Sử dụng keo nhiệt để dán gáy khi đóng quyển thay vì sử dụng băng dính và dập ghim như mô tả ở Hình A.3 Phần gáy ĐATN cần ghi các thông tin tóm tắt sau: Kỳ làm ĐATN - Ngành đào tạo - Họ và tên sinh viên - Mã số sinh viên. Ví dụ:

2022.1 - KỸ THUẬT MÁY TÍNH - NGUYỄN VĂN A - 20221234

Qui cách ghi chữ phần gáy như hình dưới đây:



Figure A.2: Qui cách đóng quyển đồ án

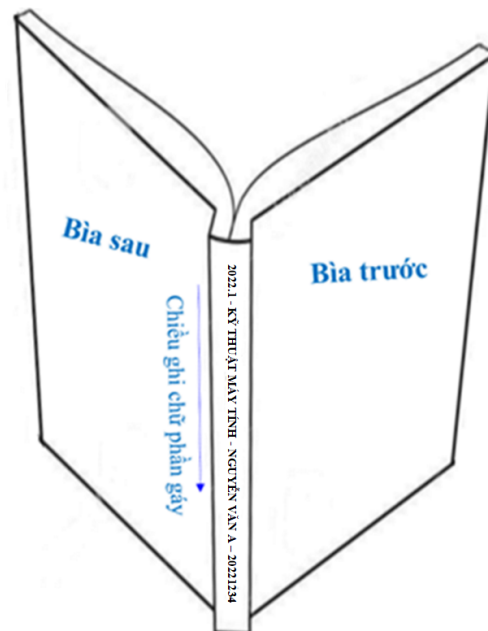


Figure A.3: Qui cách đóng quyển đồ án

B. ĐẶC TẢ USE CASE

Nếu trong nội dung chính không đủ không gian cho các use case khác (ngoài các use case nghiệp vụ chính) thì đặc tả thêm cho các use case đó ở đây.

B.1 Đặc tả use case “Thống kê tình hình mượn sách”

...

B.2 Đặc tả use case “Đăng ký làm thẻ mượn”

...